ÇANKAYA UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

CENG 407

System Architecture and Methodology

TARAS: Agricultural Analysis System

ARDA KAHRAMAN - 202111002
HATİCE ELİF ARABACI - 202211008
ÖYKÜ EYLÜL BOZDAĞ - 202311410
MUSTAFA BERK KESKİN - 202211042

TABLE OF CONTENTS

1. Introduction **3**
1.1. Purpose 3
1.2. Definitions and Acronyms 3
2. System Overview **6**
3. System Architecture **6**
3.1. Architectural Design 6
3.1.1. Layered Architecture 6
3.1.1.1. Presentation Layer 6
3.1.1.2. Application & Analytics Layer 6
3.1.1.3. Data & Cloud Services Layer 7
3.1.1.4. Device & Sensing Layer 7
3.1.1.5. Communication Layer 7
3.1.2. Client–Server Architecture 7
3.2. Decomposition Description 7
3.2.1. Field Sensing & Data Acquisition Subsystem 7
3.2.2. Analytics & Decision Support Subsystem 7
3.2.3. Visualization & User Interaction Subsystem 8
3.2.4. Class Diagram 8
3.3. System Modeling 9
3.3.1. Activity Diagrams 9
3.3.2. Sequence Diagrams 26
4. Methodology **34**
4.1. Irrigation Prediction 34
4.1.1. Problem Formulation 34
4.1.2. Data Acquisition and Preprocessing 35
4.1.3. Algorithmic Approach 35
4.1.4. System Evaluation Metrics 36
4.1.5. Implementation Details 36
4.2. Disease Detection 36
4.2.1. Problem Formulation 36
4.2.2. Data Acquisition and Preprocessing 36
4.2.3. Algorithmic Approach 37
4.2.4. Model Selection and Comparative Evaluation 37
4.2.5. System Evaluation Metrics 38
4.2.6. Implementation Details 39
4.3. LLM-Based Advisory & Decision Support 39
4.3.1. Problem Formulation 39
4.3.2. Input Representation and Context Construction 40
4.3.3. Response Generation and Explainability 40
4.3.4. System Evaluation and Limitations 40
4.3.5. Implementation Details 40
5. Data Design **41**
5.1. Entity-Relationship Diagram 41
5.2. Data Pipeline 41
6. User Interface Design **42**
6.1. Interface Hierarchy 42
6.2. Visual Presentation 42
6.3. Accessibility and Responsivity 43
7. Conclusion **44**
8. References **45**

1. Introduction

1.1. Purpose

This document defines the system architecture of the TARAS precision agriculture platform. It presents the high-level structural design of the system by describing its core architectural components, logical layers, and their interactions within the overall platform. The architectural decisions described in this document are derived directly from the functional and non-functional requirements defined in the Software Requirements Specification (SRS).

In addition to the architectural design, the document outlines the methodological approach adopted for data acquisition, preprocessing, and irrigation decision logic, explaining how agronomic principles and sensor-driven analytics are translated into operational system behavior. It also describes the role of auxiliary AI-assisted advisory components, which support explainability and user understanding without performing core decision-making.

It further explains how data flows across sensing, processing, analytics, and application layers, and how system components communicate to support scalable, reliable, and intelligent agricultural operations. The document serves as an architectural and methodological reference for guiding system implementation, integration, and future evolution, ensuring a consistent and maintainable realization of the TARAS platform.

1.2. Definitions and Acronyms

Term	**Definition**
Accuracy	The ratio of correctly classified instances (both positive and negative) to the total number of instances. It represents the overall correctness of the model's predictions.
Precision	The proportion of correctly predicted positive instances among all instances predicted as positive. It indicates how reliable the positive predictions of the model are.
Recall	The proportion of correctly predicted positive instances among all actual positive instances. It measures the model's ability to identify all relevant positive cases.
F1-Score	The harmonic mean of precision and recall. It provides a balanced measure that considers both false positives and false negatives, especially useful when class distributions are imbalanced.
Agronomy	The branch of agricultural science that focuses on crop production and soil management, examining the interactions between crops, soil, climate, and agricultural practices in order to optimize yield and resource efficiency.

Evapotranspiration (ET_0)	ET_0 is a standardized measure of atmospheric water demand, representing the evapotranspiration rate from a hypothetical reference surface under given meteorological conditions. It reflects the effect of weather conditions on water loss independently of crop type.
Crop Coefficients(Kc)	Kc is a dimensionless coefficient used to adjust reference evapotranspiration (ET_0) to account for crop type and growth-stage characteristics, enabling estimation of crop-specific water demand.
JWT(JSON Web Token)	JWT is a compact and secure token format used for authentication and authorization, allowing the system to verify user identity and access rights between the client and backend services.
ML	ML refers to computational techniques that enable systems to learn patterns from data and make predictions or decisions without being explicitly programmed, and is used in the TARAS system primarily for plant disease detection and auxiliary analysis tasks.
CV Models	CV models are computational models designed to analyze and interpret visual data such as images, enabling tasks like object recognition, classification, and pattern detection. In the TARAS system, they are used for plant leaf disease detection.
Cloud Services	Cloud services refer to on-demand computing resources such as storage, databases, and backend services provided over the internet, used in the TARAS system to support data storage, processing, and system scalability.
Client	The client is the user-facing component of a client–server architecture that initiates requests to the server and presents system functionality and data to the user. In the TARAS system, the mobile application acts as the client.
Server	The server is the backend component of a client–server architecture that receives requests from clients, processes them, manages data storage and system logic, and returns responses. In the TARAS system, the server hosts backend services, databases, and analysis modules.
Storage Bucket	A storage bucket is a logical container in cloud storage systems used to store and organize unstructured data such as images and files. In the TARAS system, storage buckets are used to store uploaded plant images and related assets.
Classification	Classification is a ML task in which input data are assigned to one of several predefined categories or classes. In the TARAS system, classification is used to identify plant diseases from leaf images.
Processing Pipeline	The process of collecting raw data from heterogeneous sources and transforming it into a consistent, reliable, and analysis-ready format. This includes acquiring environmental sensor data, historical records, metadata, and image data, followed by preprocessing steps such as timestamp alignment, unit normalization, quality control, outlier detection, and missing data handling to ensure data integrity before model-based analysis.

Deep Learning	A subfield of ML that utilizes neural networks with multiple layers to automatically learn hierarchical feature representations from data. It is particularly effective for complex tasks such as image recognition, natural language processing, and speech analysis.
CNN	A deep learning architecture designed to automatically extract features from image data using convolutional layers. It is widely used in tasks such as image classification and object recognition.
Overfitting	Overfitting refers to a modeling phenomenon where a ML model learns training data too closely, resulting in high training performance but poor generalization to unseen data.
Pre-trained weights	Model parameters obtained by training a neural network on a large-scale dataset prior to being applied to a target task. They are used to accelerate training and improve generalization performance.
Finetune	The process of retraining some or all layers of a pre-trained model on a task-specific dataset using a low learning rate, allowing the model to adapt to the new problem domain.
TensorFlow	An open-source ML framework developed by Google for building, training, and deploying ML and deep learning models.
Keras	A high-level deep learning API that runs on top of TensorFlow, enabling fast and efficient development of neural network models with a user-friendly and modular structure.
WCAG (Web Content Accessibility Guidelines)	WCAG is a set of international guidelines that define standards for making digital interfaces accessible to users with diverse abilities, focusing on principles such as perceivability, operability, understandability, and robustness.
Cognitive Load	Cognitive load refers to the amount of mental effort required to process information and interact with a system. In the TARAS interface design, reducing cognitive load aims to make the application easier to understand and use, especially for users with limited technical experience.
DAP	DAP refers to the number of days elapsed since a crop was planted and is used to represent crop growth stages for time-dependent analyses such as crop coefficient (Kc) estimation and irrigation planning.

2. System Overview

The TARAS (Tarım Analizi Sistemi) platform is a precision agriculture decision-support system designed to assist farmers in irrigation planning, crop health monitoring and sustainability assessment. The project initially explored a wide range of smart agriculture features, which were evaluated based on impact, feasibility, usability, cost and sustainability. Based on literature analysis and comparisons with existing solutions, the system scope was refined to focus on the most effective and implementable components. The final design centers on integrated field sensing, AI-driven analysis, digital twin visualization and a mobile application interface, ensuring a practical, scalable and user-oriented solution.

3. System Architecture

3.1. Architectural Design

The architectural design of the TARAS follows a hybrid architecture that combines Layered Architecture and Client–Server Architecture principles. This approach supports modular development, scalability and clear separation of responsibilities across system components.

3.1.1. Layered Architecture

3.1.1.1. Presentation Layer

Provides the mobile application interface through which farmers interact with the system. This layer presents sensor readings, irrigation recommendations, disease detection results, digital twin visualizations, sustainability metrics, and AI-assisted advisory explanations in an accessible and user-friendly manner. The advisory interface translates analytical outputs into clear, context-aware guidance, supporting user understanding without replacing farmer decision-making.

3.1.1.2. Application & Analytics Layer

Implements the core system logic, including data processing, irrigation prediction, disease analysis, decision-support workflows and user interaction management. ML and computer vision models operate within this layer to transform raw data into actionable insights.

3.1.1.3. Data & Cloud Services Layer

Handles data storage, retrieval and synchronization between field devices and the mobile application. This layer manages time-series sensor data, historical records, model outputs and user inputs using a centralized cloud-based backend.

3.1.1.4. Device & Sensing Layer

Includes IoT sensor nodes deployed in the field, responsible for collecting soil moisture, temperature and environmental data. This layer abstracts hardware-level interactions and ensures reliable data acquisition.

3.1.1.5. Communication Layer

Manages data transmission between sensor nodes, gateway devices and cloud services using long-range and internet-based communication protocols. It ensures reliable, low-power and fault-tolerant data flow across the system.

3.1.2. Client–Server Architecture

The TARAS mobile application operates as a client that interacts with the cloud-based backend services. Field sensor data, user inputs, and system outputs, such as irrigation recommendations, disease analysis results and sustainability metrics are exchanged through request–response communication mechanisms. This separation between client and server components enables loose coupling, supports scalability and simplifies maintenance and future extensions.

3.2. Decomposition Description

3.2.1. Field Sensing & Data Acquisition Subsystem

- Soil Moisture Sensing Module
- Environmental Data Collection (Temperature, Humidity)
- Sensor Calibration & Validation Module
- Gateway Communication Module

3.2.2. Analytics & Decision Support Subsystem

- Data Storage Module
- Irrigation Prediction Module
- Disease Detection Module
- Sustainability & Carbon Footprint Calculation Module
- Data Processing & Model Management Module
- LLM advisory & Explainability Module

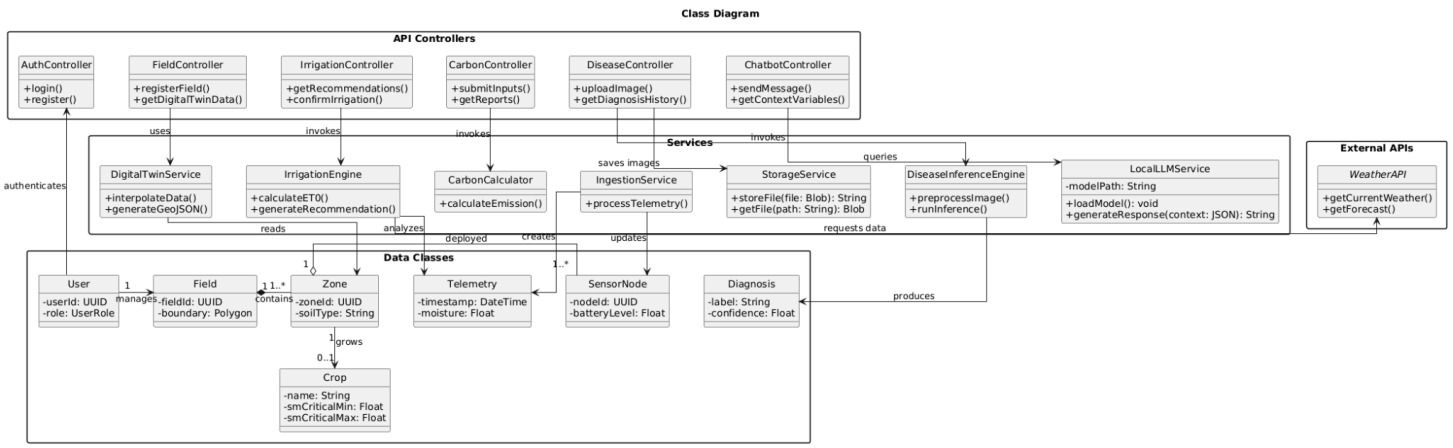
3.2.3. Visualization & User Interaction Subsystem

- Mobile Application Interface
- Digital Twin Visualization Module
- Recommendation & Alert Presentation
- User Input & Configuration Management

Each module in the TARAS platform interacts with other components through clearly defined interfaces, encapsulating internal logic and exposing only essential operations such as retrieving sensor measurements, generating irrigation recommendations, performing disease analysis, or updating visualization data. This approach supports abstraction, modular refinement, and reusability in line with core software engineering design principles.

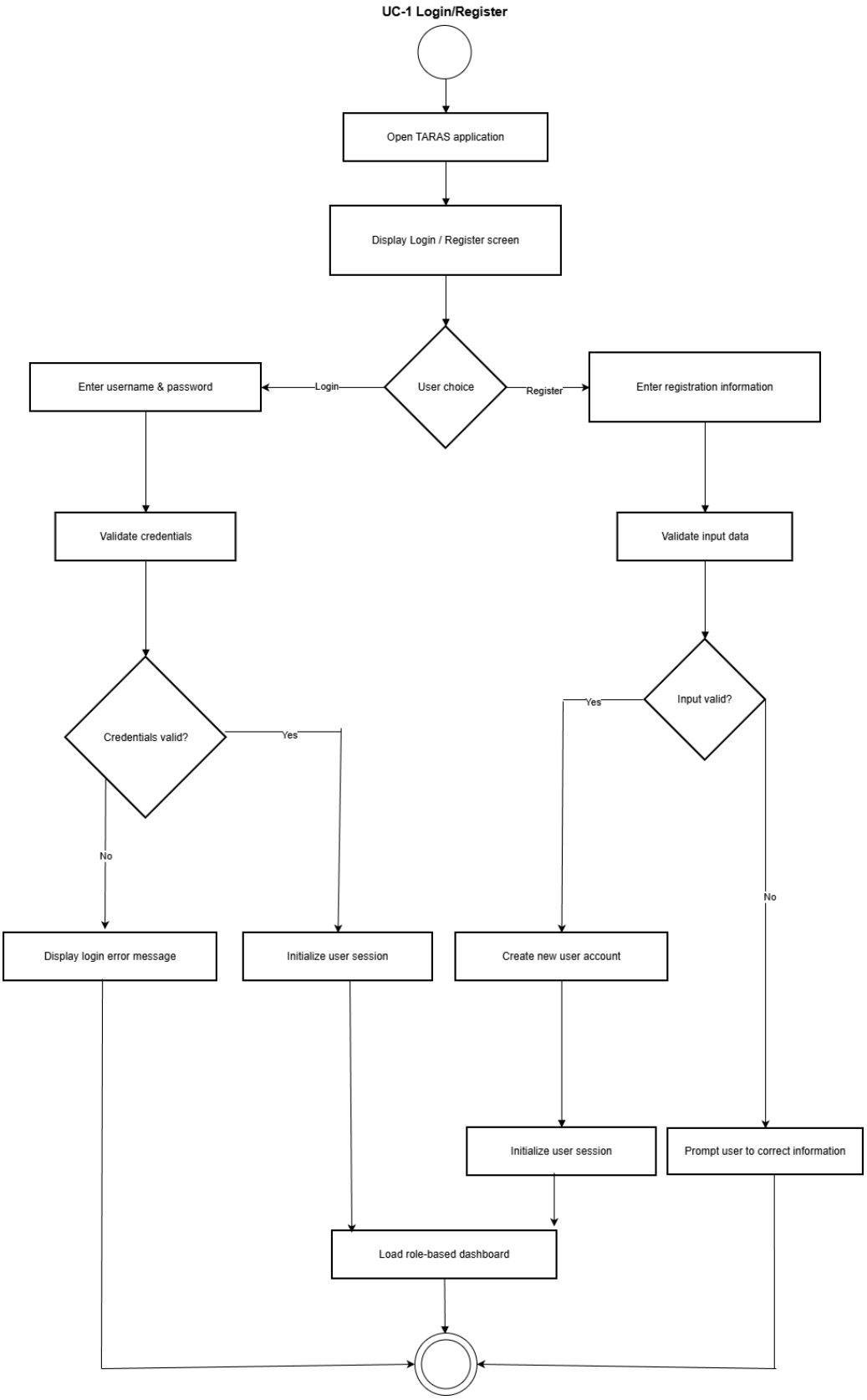
The modular system structure simplifies independent development and testing of components and allows future extensions, such as integrating new sensor types, improving prediction models or expanding sustainability metrics without impacting the overall system architecture.

3.2.4. Class Diagram



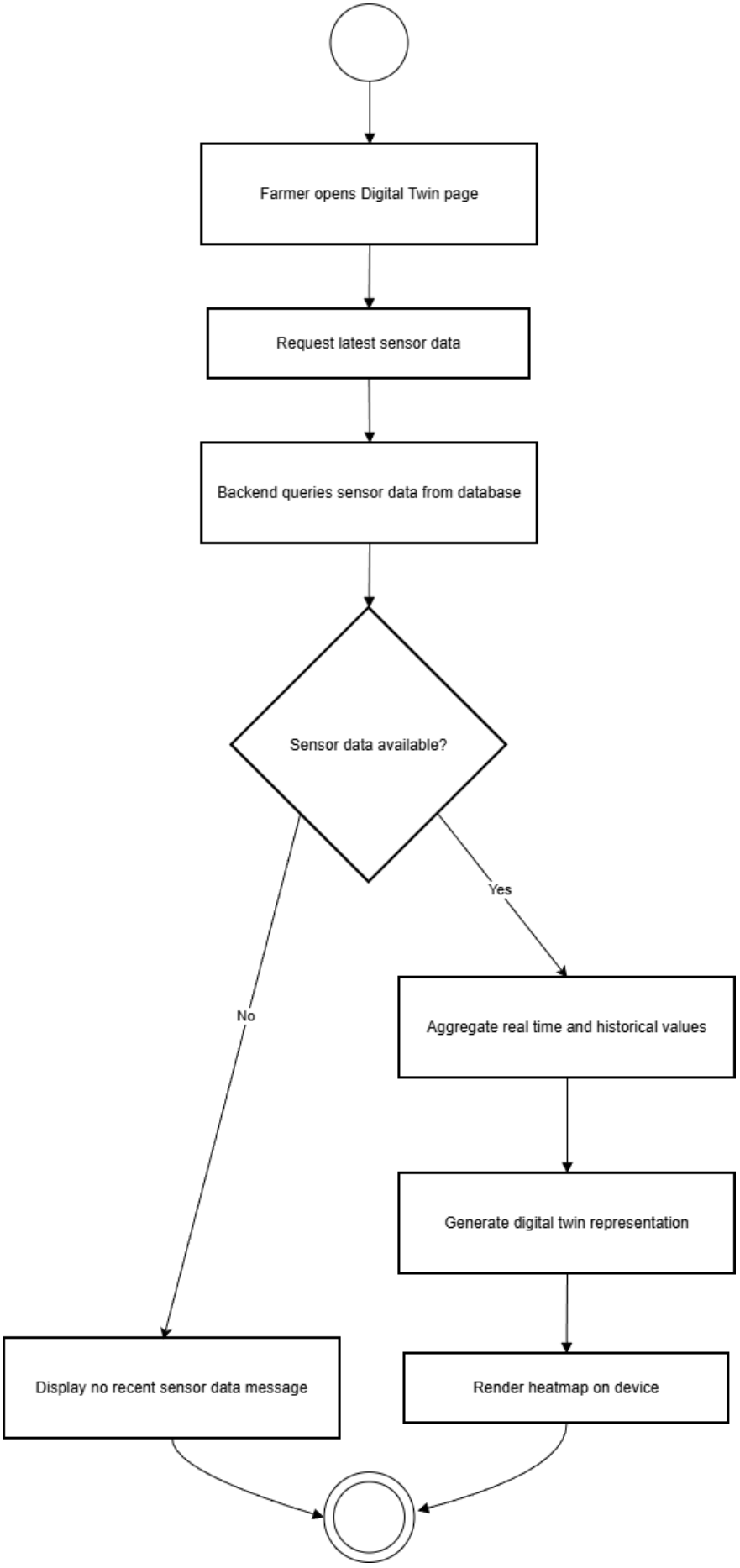
3.3. System Modeling

3.3.1. Activity Diagrams

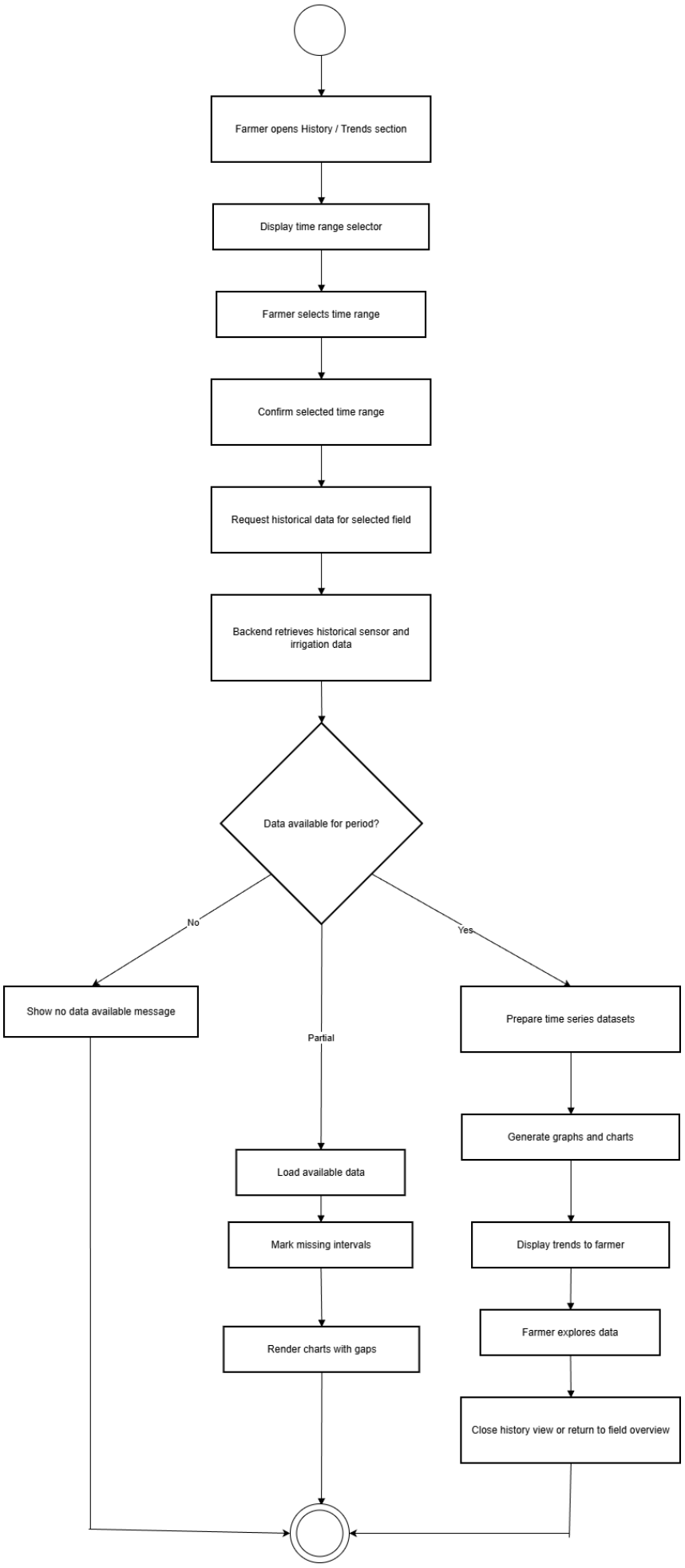


UC-2 New Field Registration
Farmer logged in
Open New Field Registration page
Scan for nearby sensor nodes
or retrieve available node IDs
Sensor nodes detected?
No
Yes
Notify farmer: No sensor nodes found
Display available sensor node IDs
Farmer selects sensor nodes
Validate selected node IDs
IDs valid and unique?
No
Yes
Request correction from farmer
Create new field record
Link selected sensor nodes to field
Confirm field registration

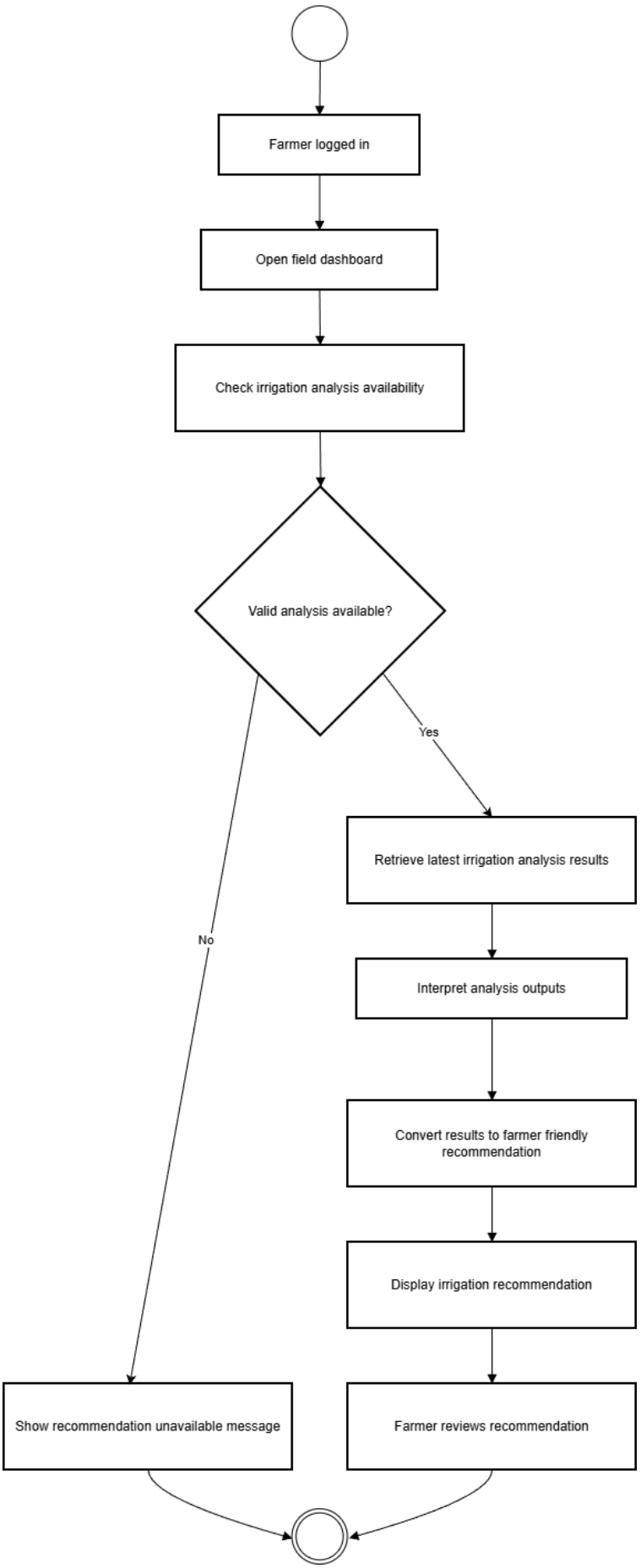
UC-3 View Digital Twin

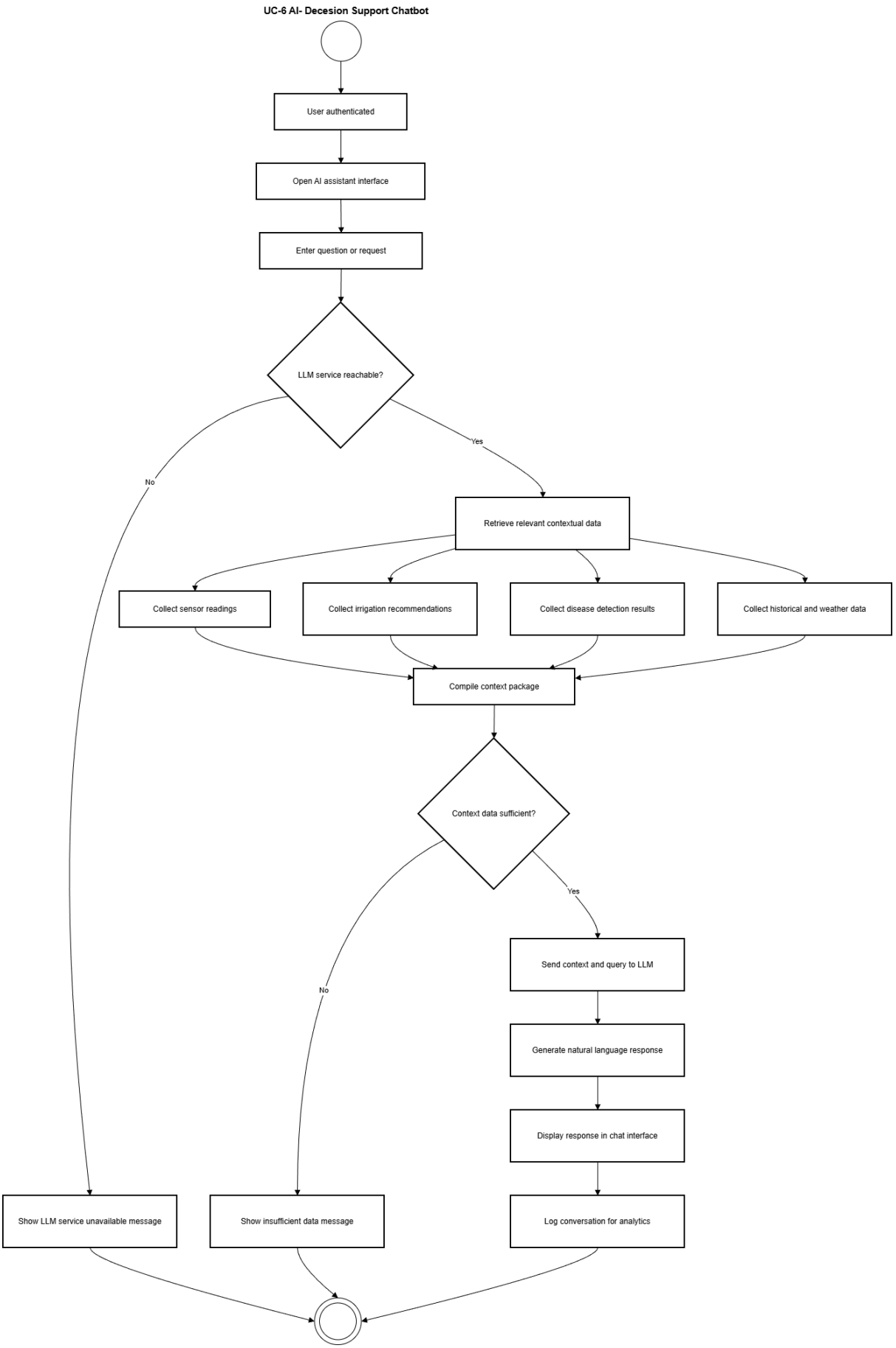


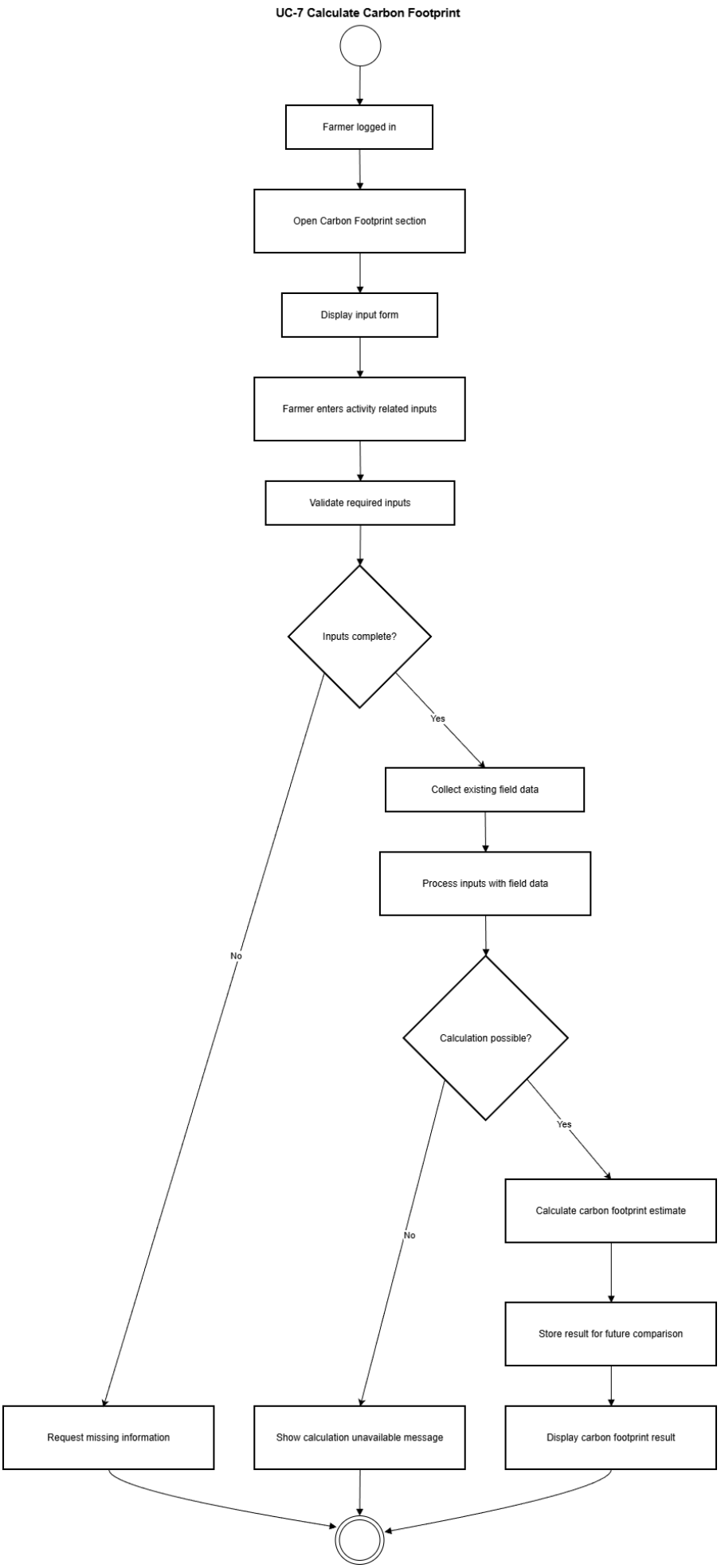
UC-4 View Historical Data/Trends



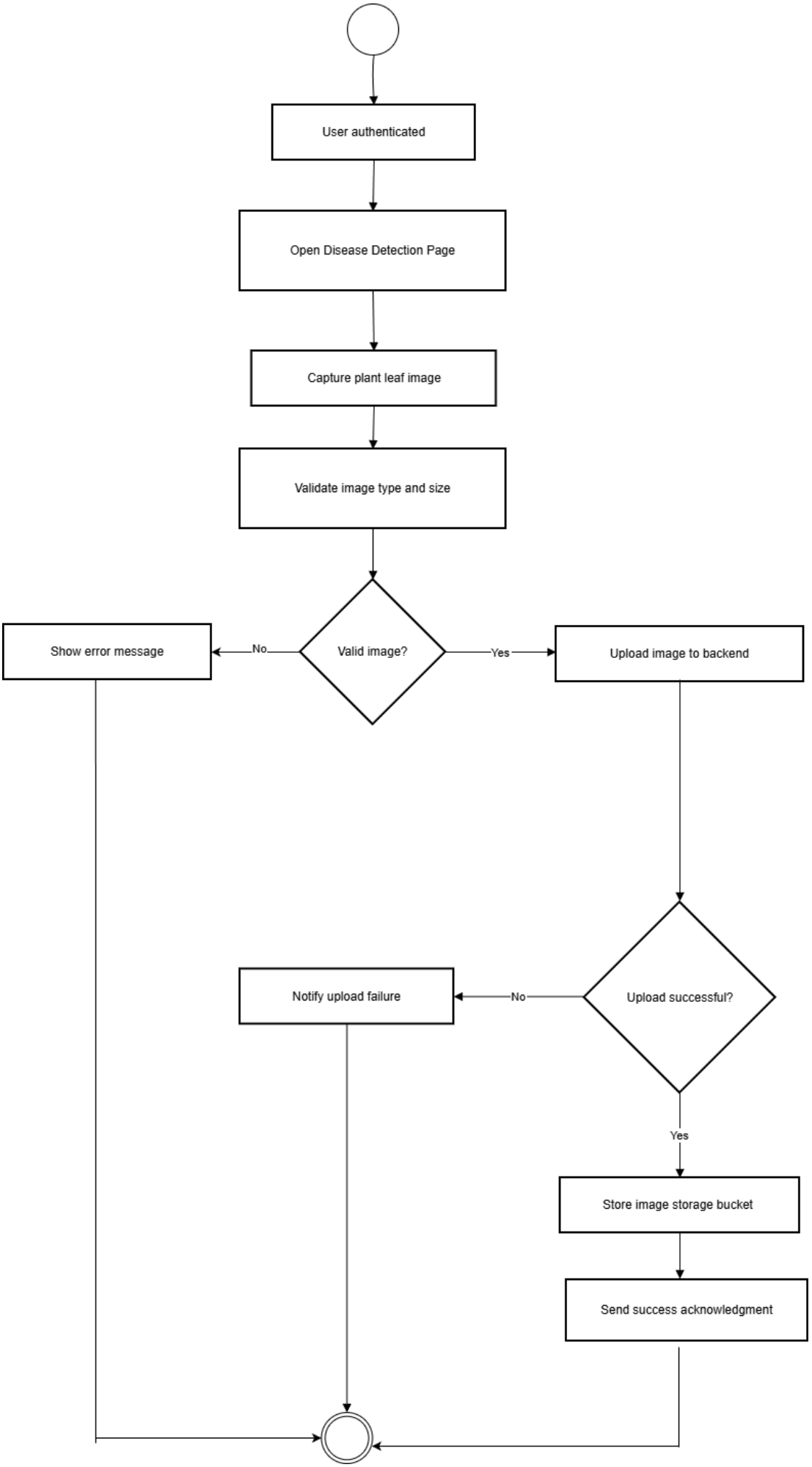
UC-5 Receive Irrigation Suggestion



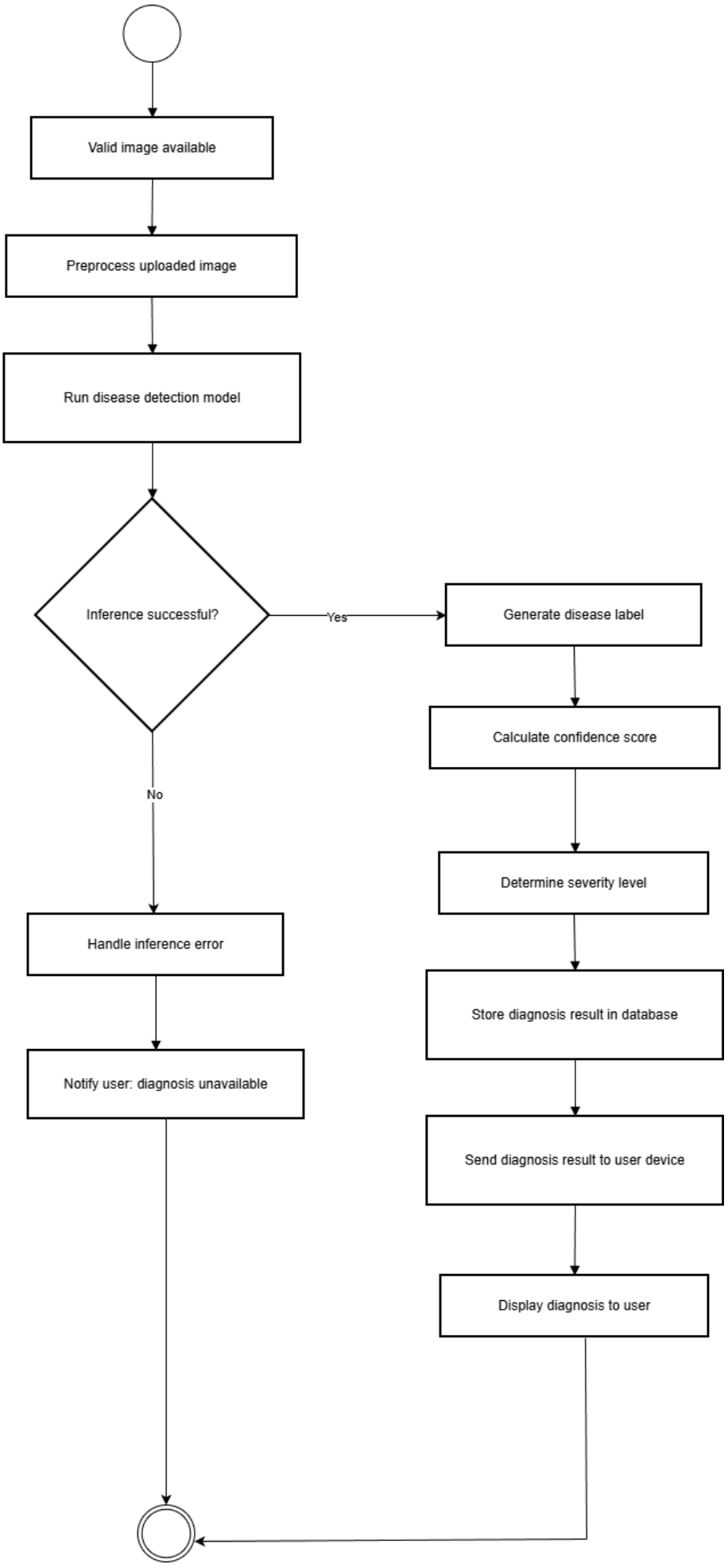

UC-6 AI- Decesion Support Chatbot
User authenticated
Open AI assistant interface
Enter question or request
LLM service reachable?
Yes
No
Retrieve relevant contextual data
Collect sensor readings
Collect irrigation recommendations
Collect disease detection results
Collect historical and weather data
Compile context package
Context data sufficient?
Yes
No
Send context and query to LLM
Generate natural language response
Display response in chat interface
Log conversation for analytics
Show LLM service unavailable message
Show insufficient data message


UC-7 Calculate Carbon Footprint
Farmer logged in
Open Carbon Footprint section
Display input form
Farmer enters activity related inputs
Validate required inputs
Inputs complete?
Yes
No
Collect existing field data
Process inputs with field data
Calculation possible?
Yes
No
Calculate carbon footprint estimate
Store result for future comparison
Request missing information
Show calculation unavailable message
Display carbon footprint result

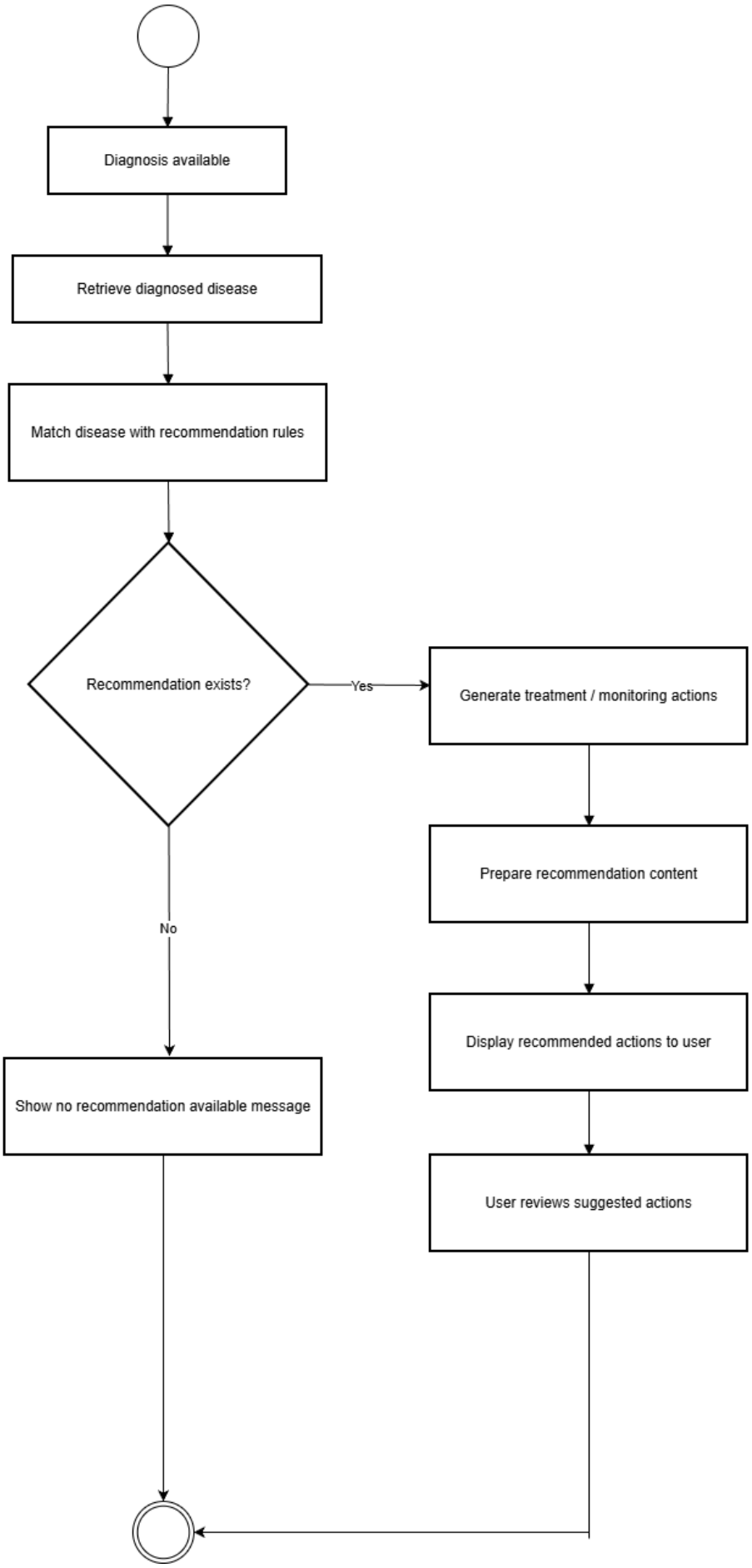
UC-8 Upload Plant Image

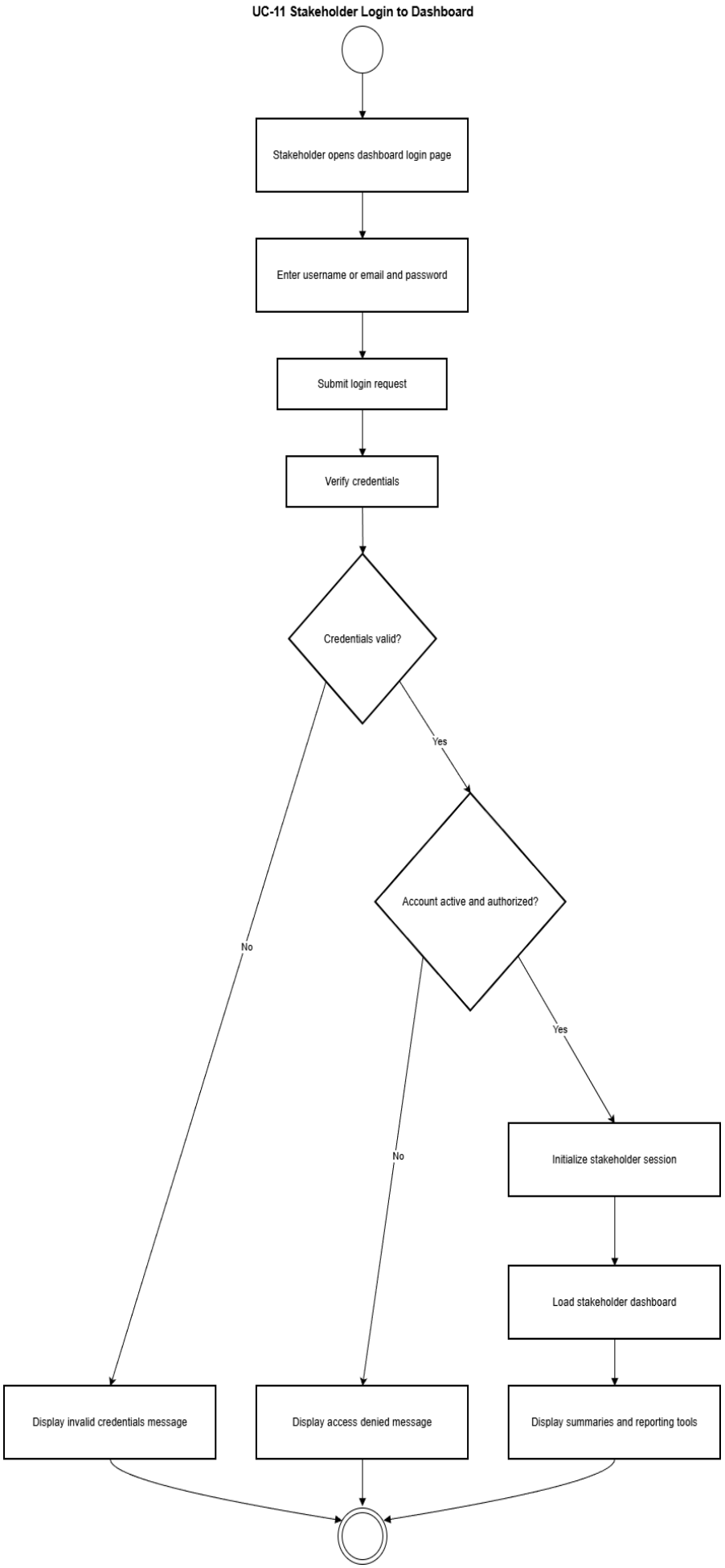


UC-9 Receive Disease Diagnosis

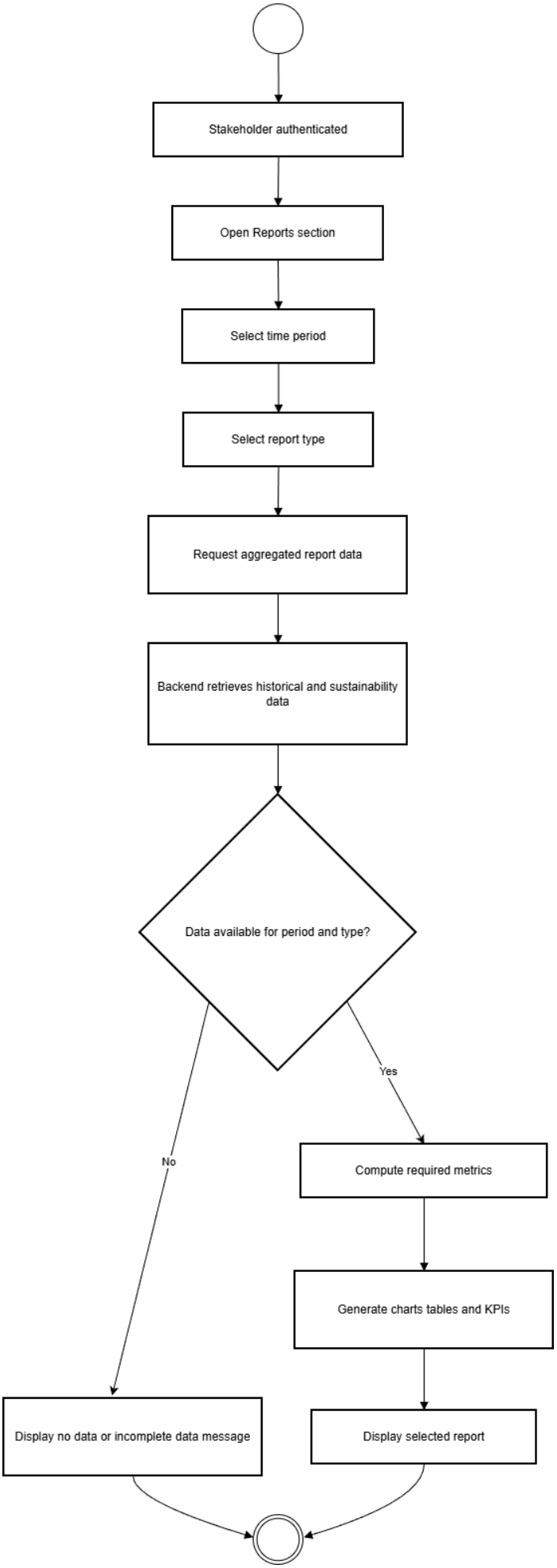


UC-10 View Recommended Actions

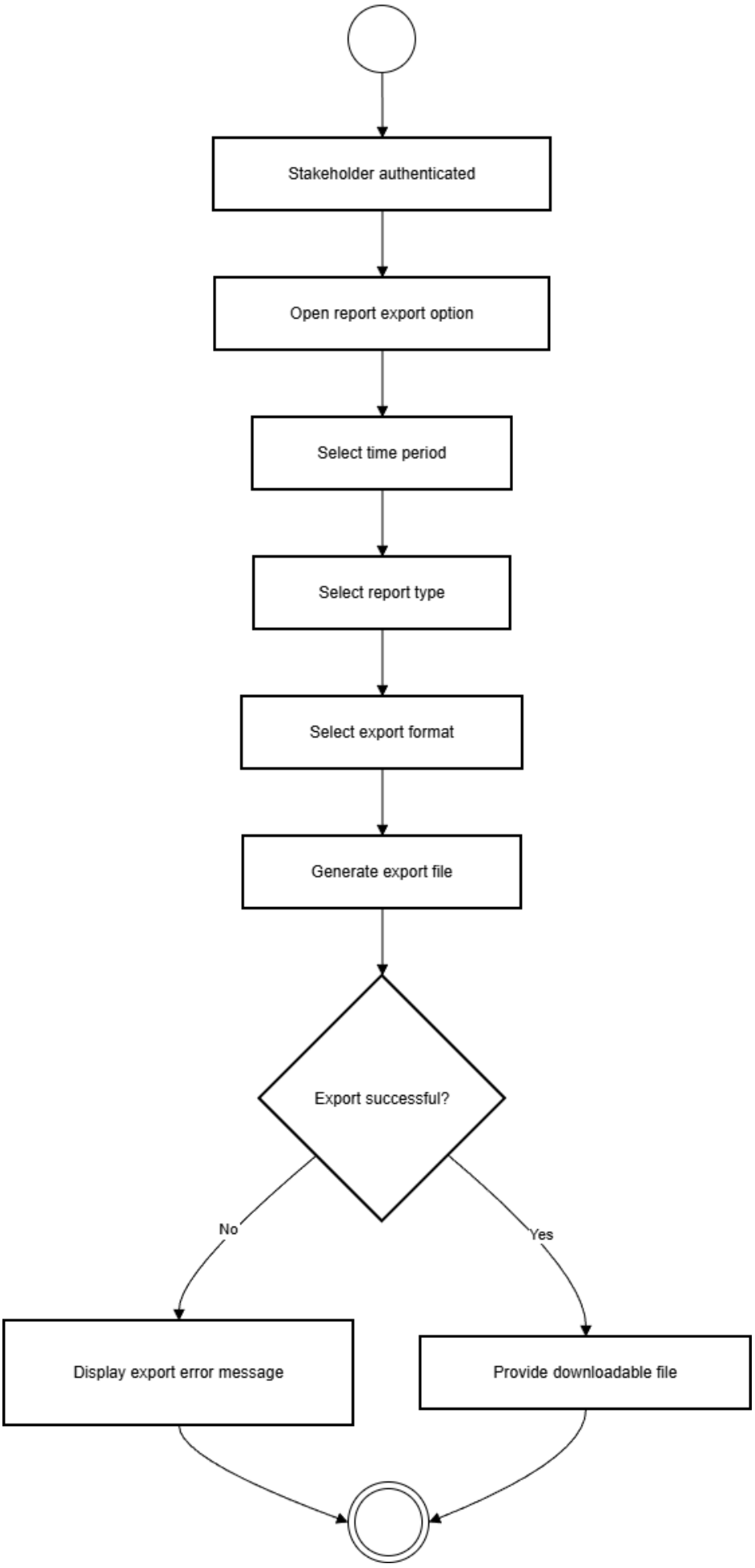


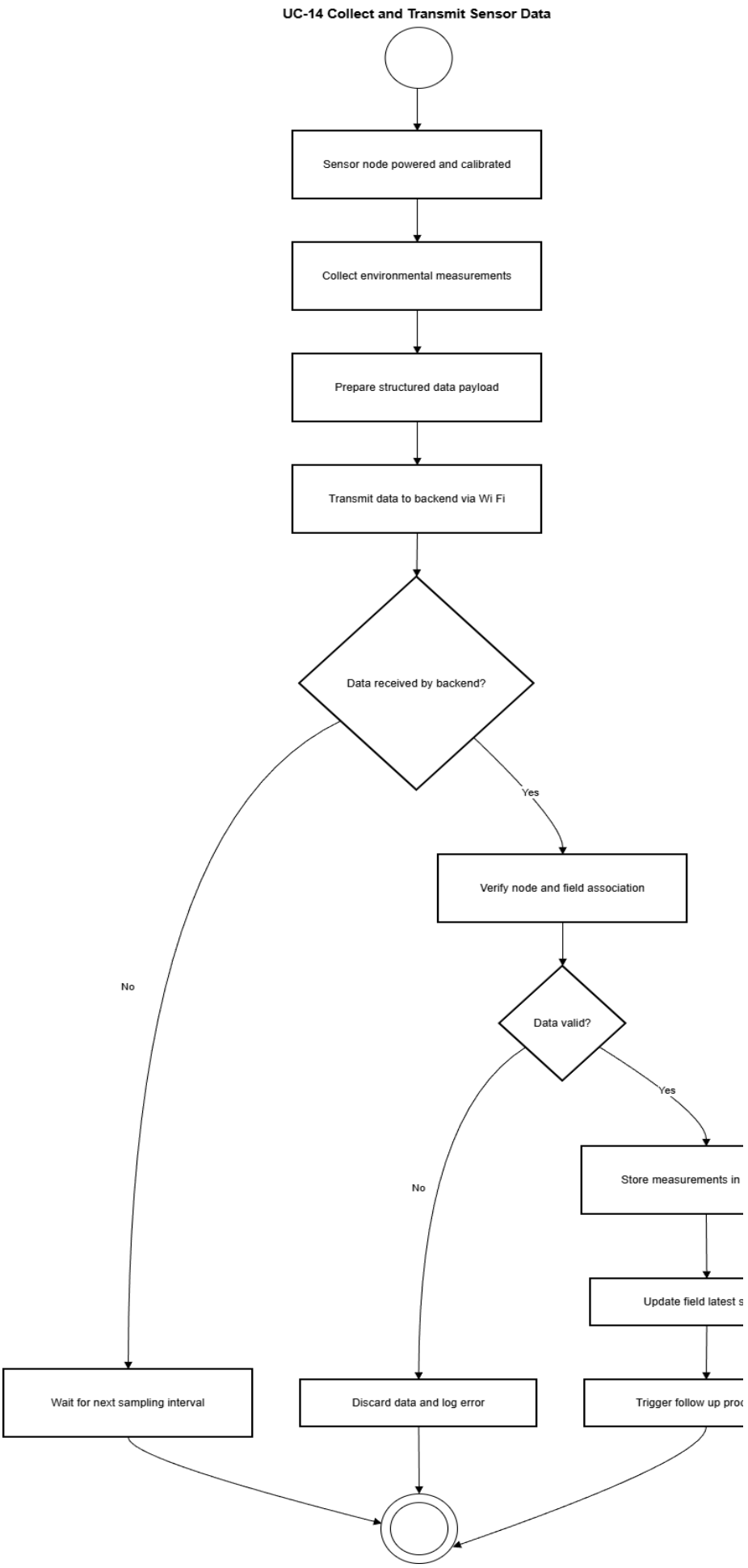

UC-11 Stakeholder Login to Dashboard
Stakeholder opens dashboard login page
Enter username or email and password
Submit login request
Verify credentials
Credentials valid?
Yes
No
Account active and authorized?
Yes
No
Initialize stakeholder session
Load stakeholder dashboard
Display invalid credentials message
Display access denied message
Display summaries and reporting tools

UC-12 View Sustainability and Carbon Footprint Reports

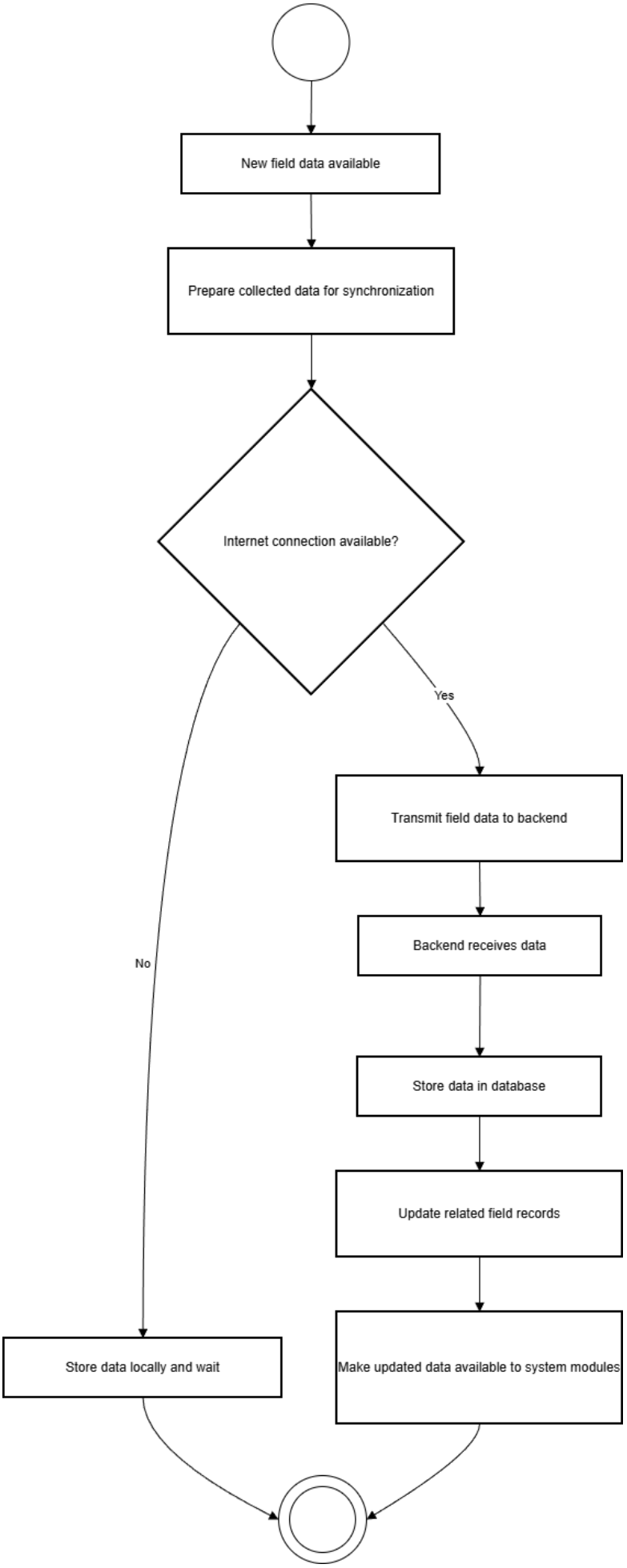


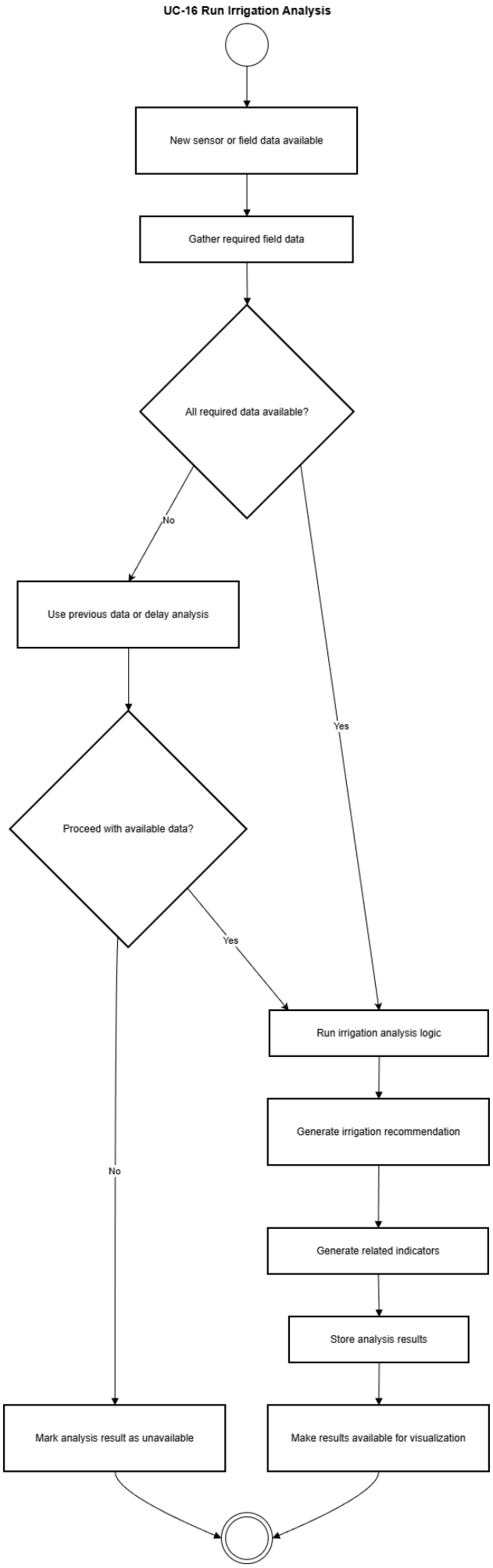
UC-13 Export Sustainability and Carbon Footprint Reports



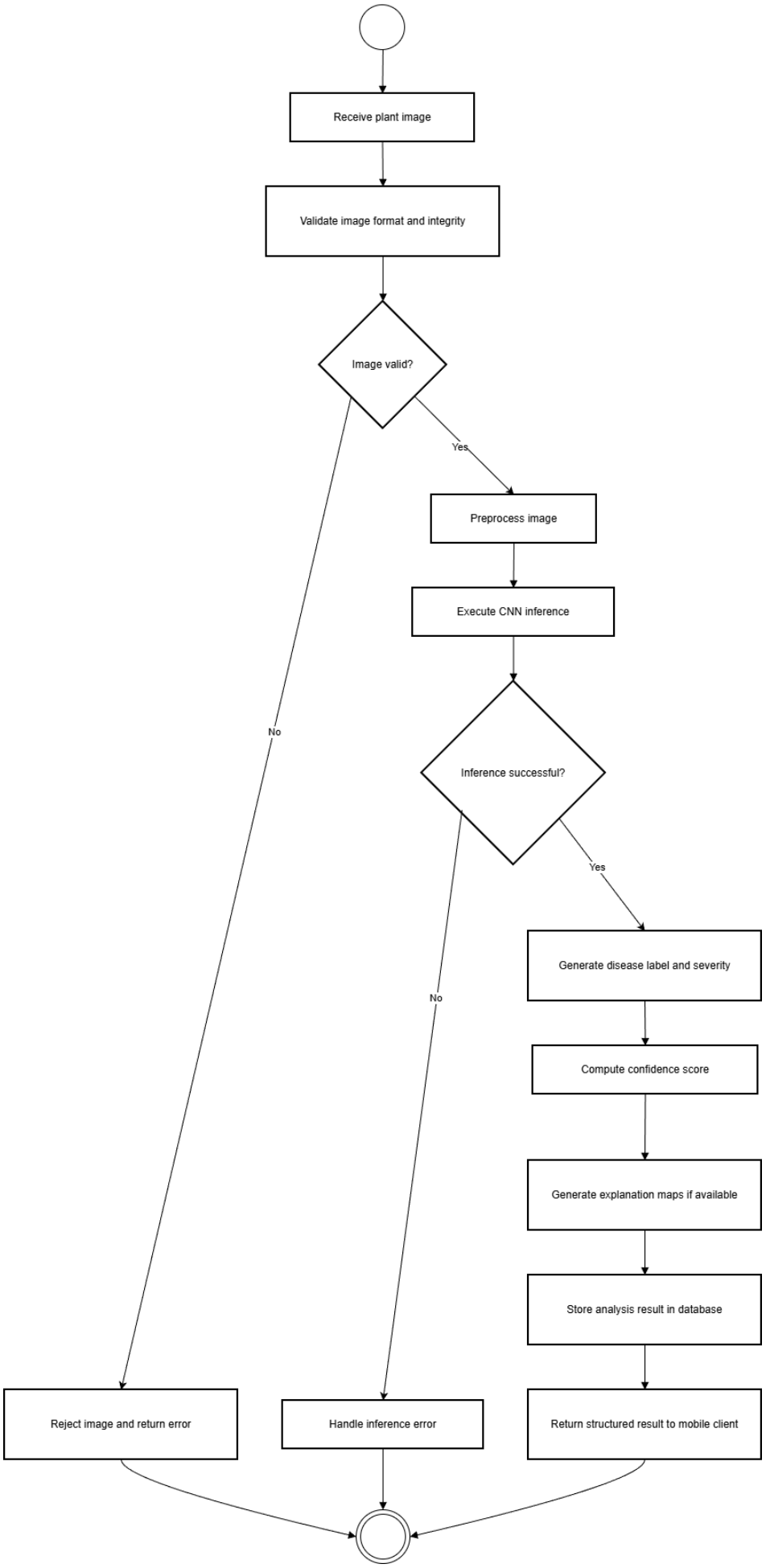

UC-14 Collect and Transmit Sensor Data
Sensor node powered and calibrated
Collect environmental measurements
Prepare structured data payload
Transmit data to backend via Wi Fi
Data received by backend?
Yes
No
Verify node and field association
Data valid?
Yes
No
Store measurements in
Update field latest s
Wait for next sampling interval
Discard data and log error
Trigger follow up proc

UC-15 Sync Field Data to Backend

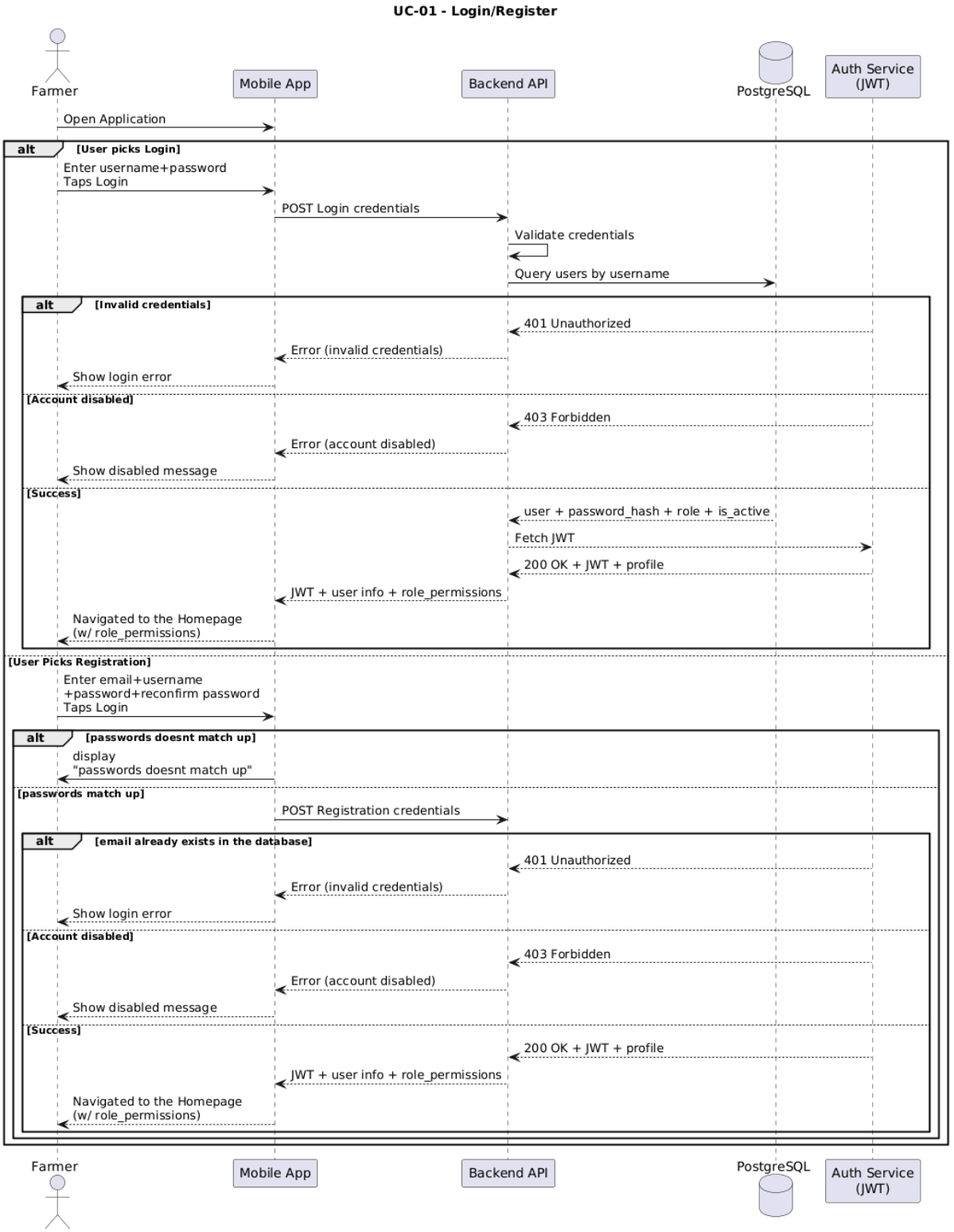


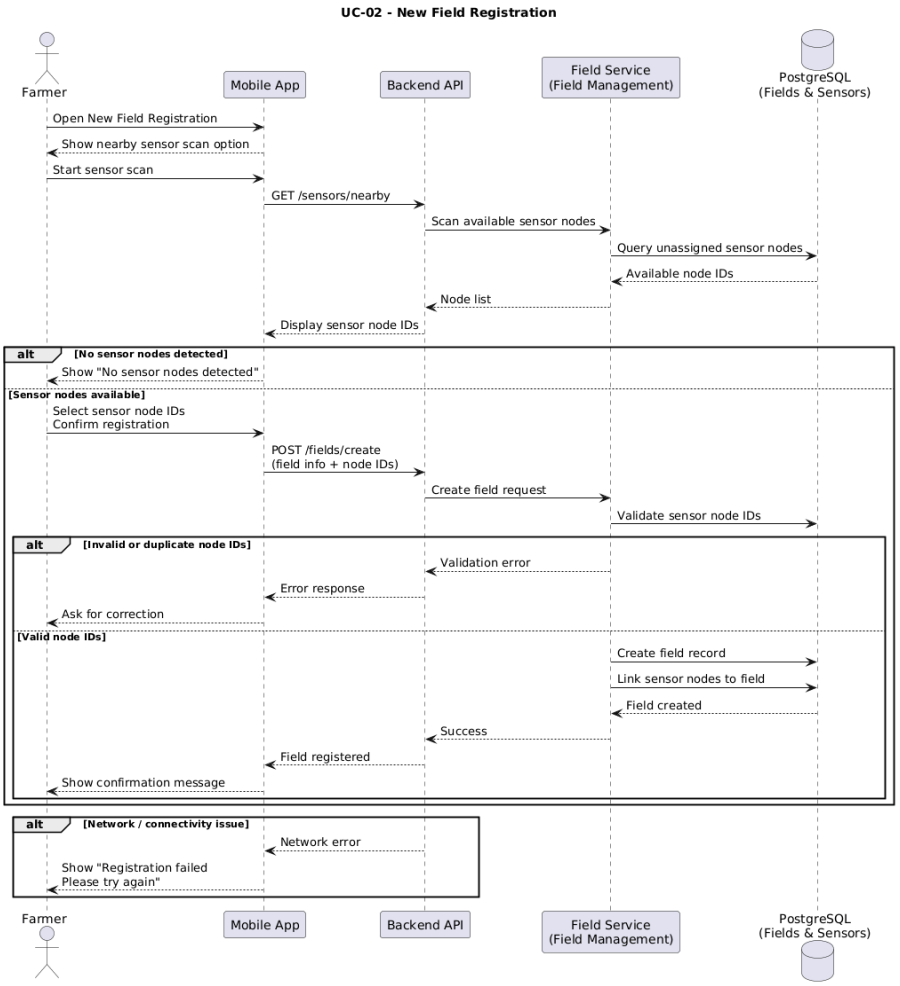

UC-16 Run Irrigation Analysis
New sensor or field data available
Gather required field data
All required data available?
No
Yes
Use previous data or delay analysis
Proceed with available data?
Yes
No
Run irrigation analysis logic
Generate irrigation recommendation
Generate related indicators
Store analysis results
Mark analysis result as unavailable
Make results available for visualization

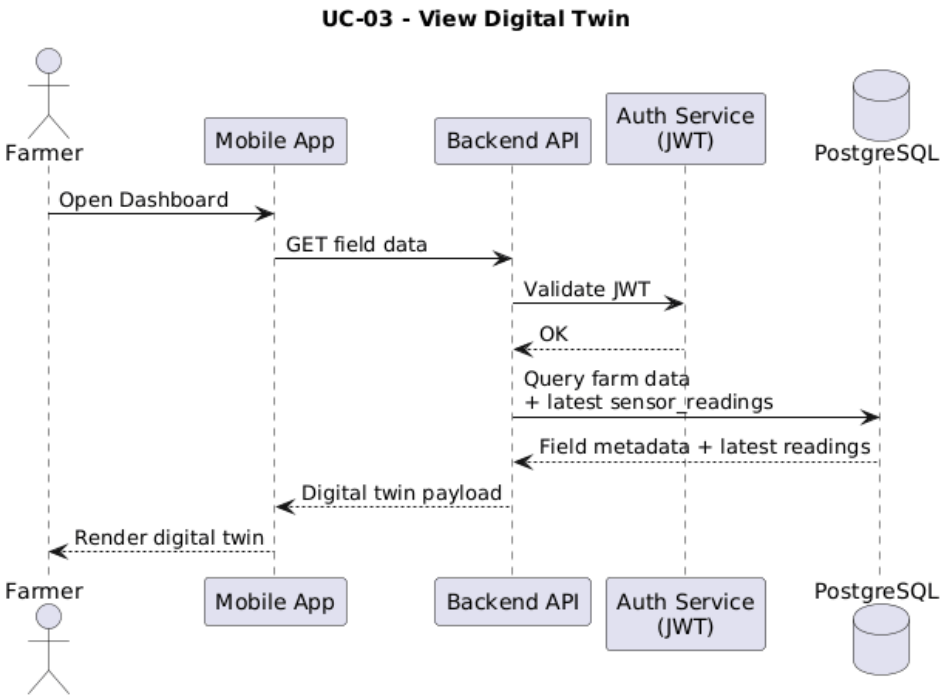
UC-17 Analyze Plant Image for Diseases



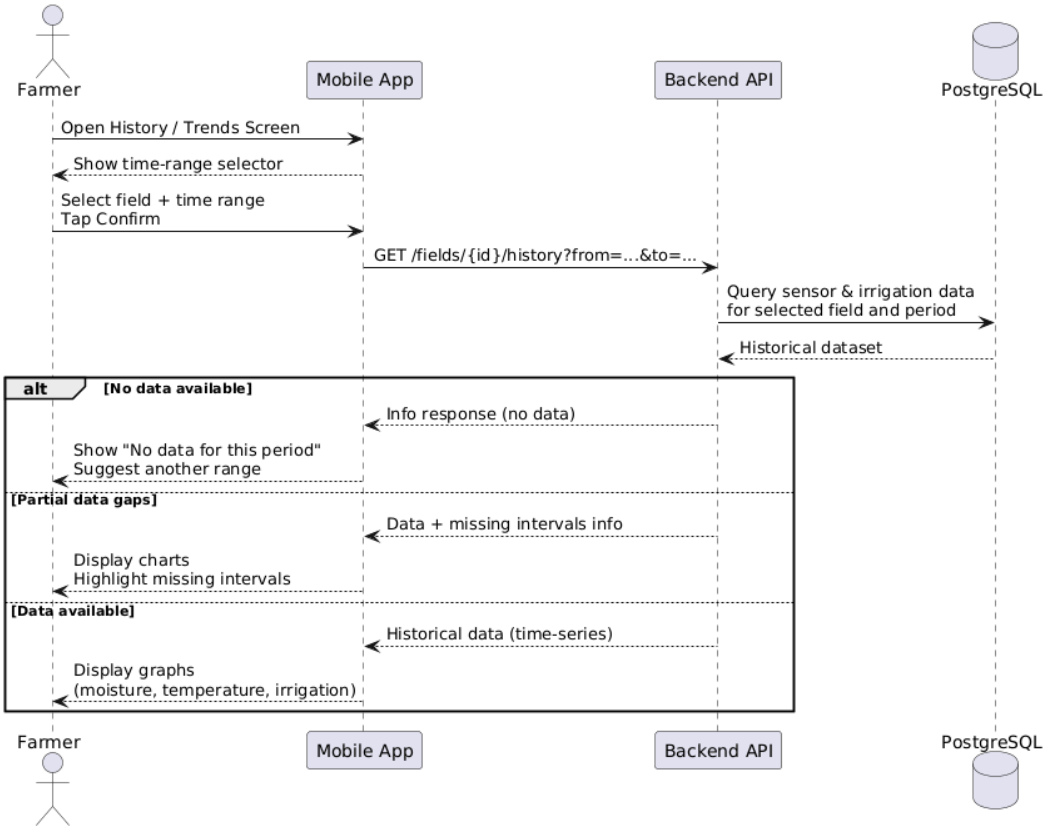
3.3.2. Sequence Diagrams



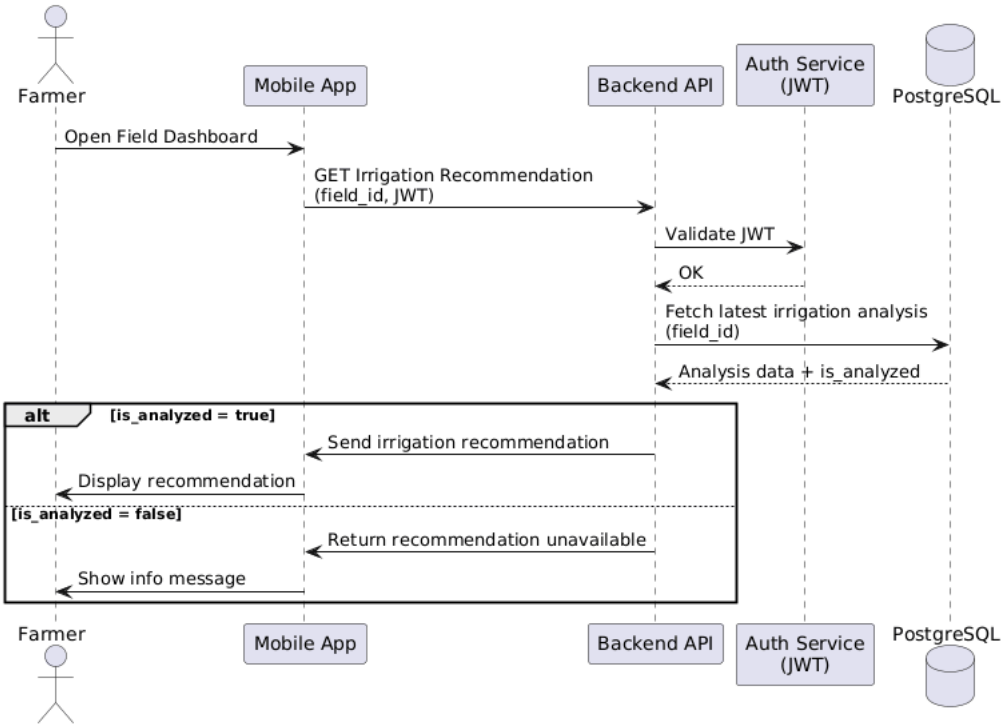

UC-02 - New Field Registration
Farmer
Mobile App
Backend API
Field Service
(Field Management)
PostgreSQL
(Fields & Sensors)
Open New Field Registration
Show nearby sensor scan option
Start sensor scan
GET /sensors/nearby
Scan available sensor nodes
Query unassigned sensor nodes
Available node IDs
Node list
Display sensor node IDs
alt
[No sensor nodes detected]
Show "No sensor nodes detected"
[Sensor nodes available]
Select sensor node IDs
Confirm registration
POST /fields/create
(field info + node IDs)
Create field request
Validate sensor node IDs
alt
[Invalid or duplicate node IDs]
Validation error
Error response
Ask for correction
[Valid node IDs]
Create field record
Link sensor nodes to field
Field created
Success
Field registered
Show confirmation message
alt
[Network / connectivity issue]
Network error
Show "Registration failed
Please try again"


UC-03 - View Digital Twin
Farmer
Mobile App
Backend API
Auth Service
(JWT)
PostgreSQL
Open Dashboard
GET field data
Validate JWT
OK
Query farm data
+ latest sensor readings
Field metadata + latest readings
Digital twin payload
Render digital twin

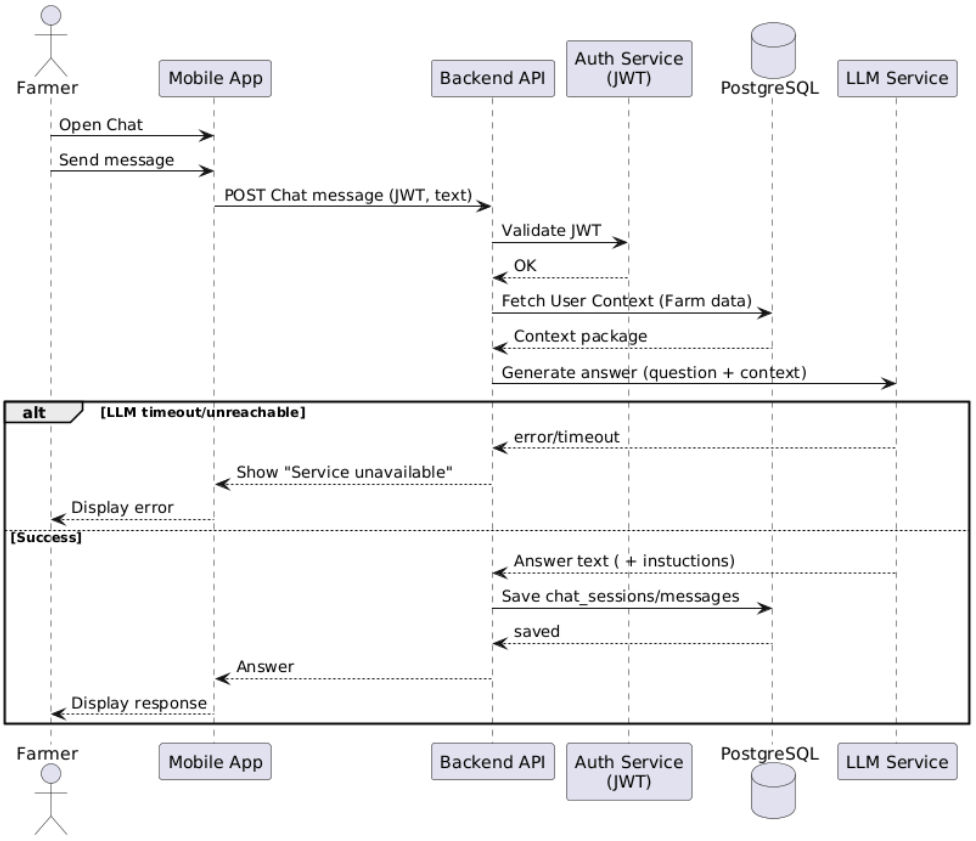
UC-04 - View Historical Data / Trends



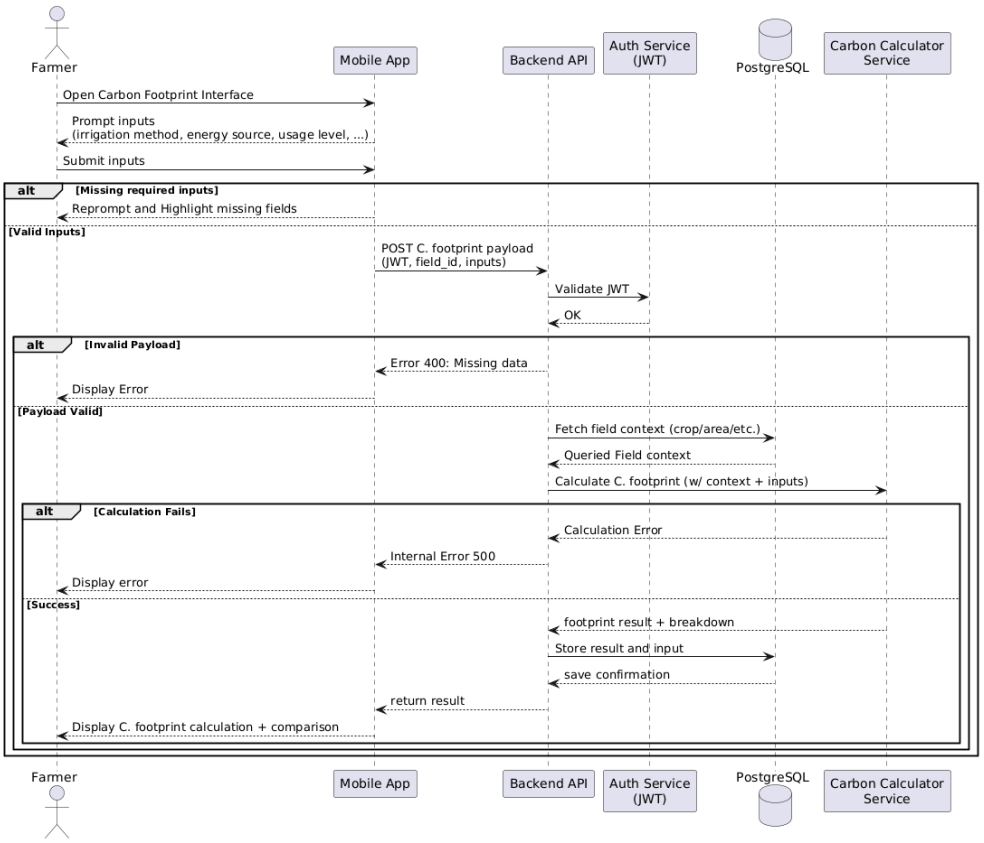
UC-05 - Receive Irrigation Suggestion



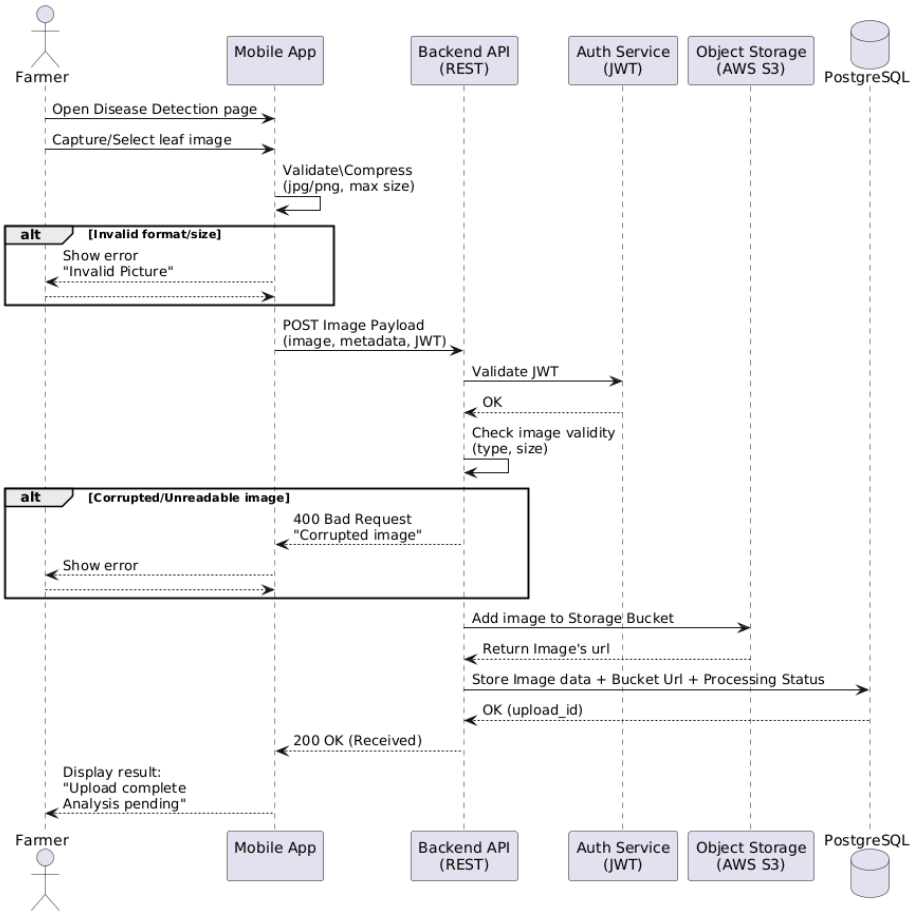
UC-06 - AI Decision-Support Chatbot



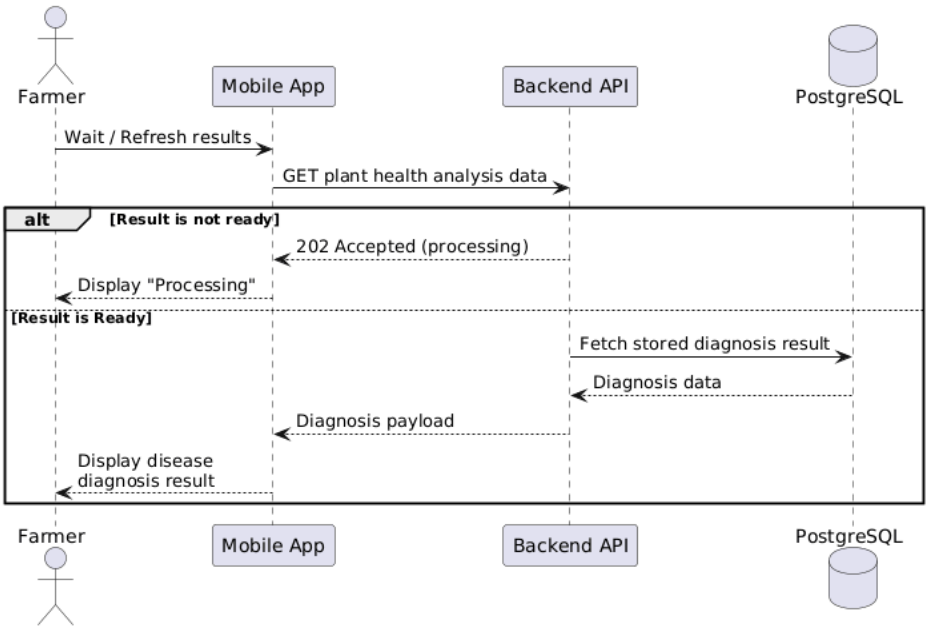
UC-07 - Calculate Carbon Footprint

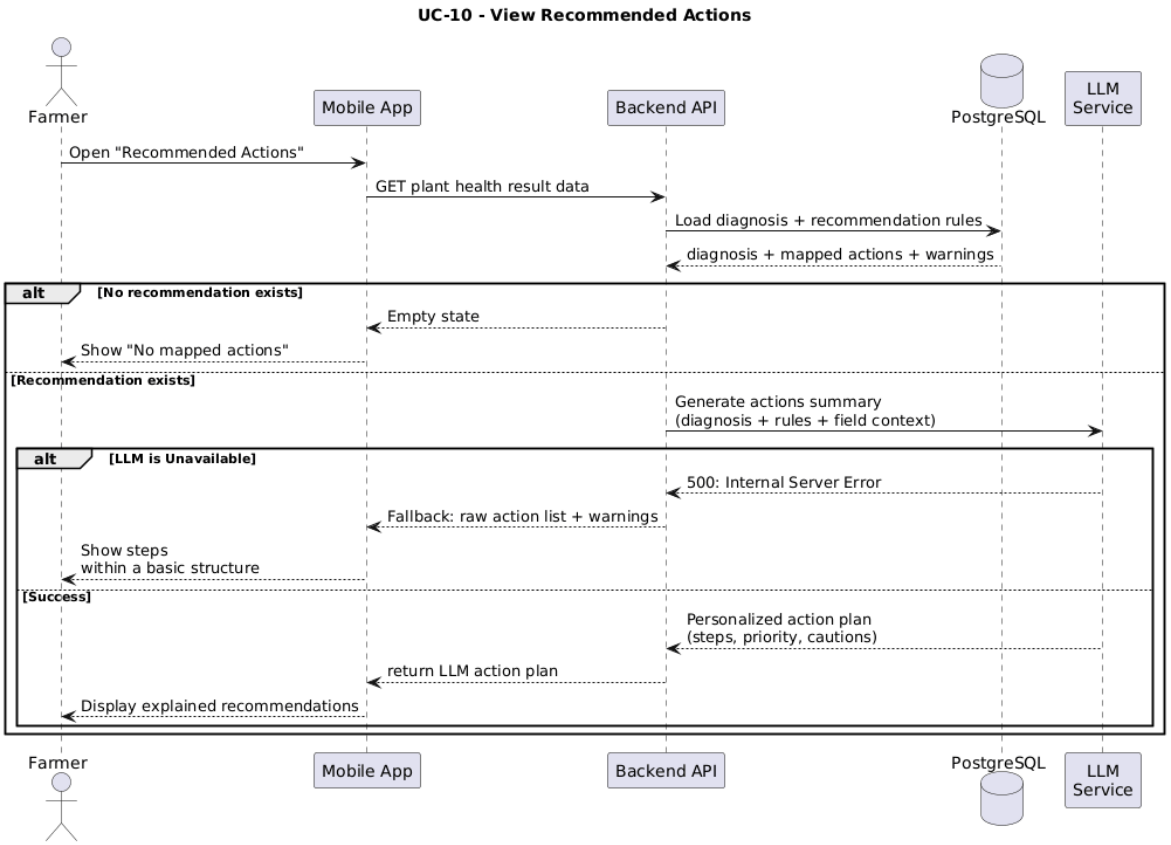


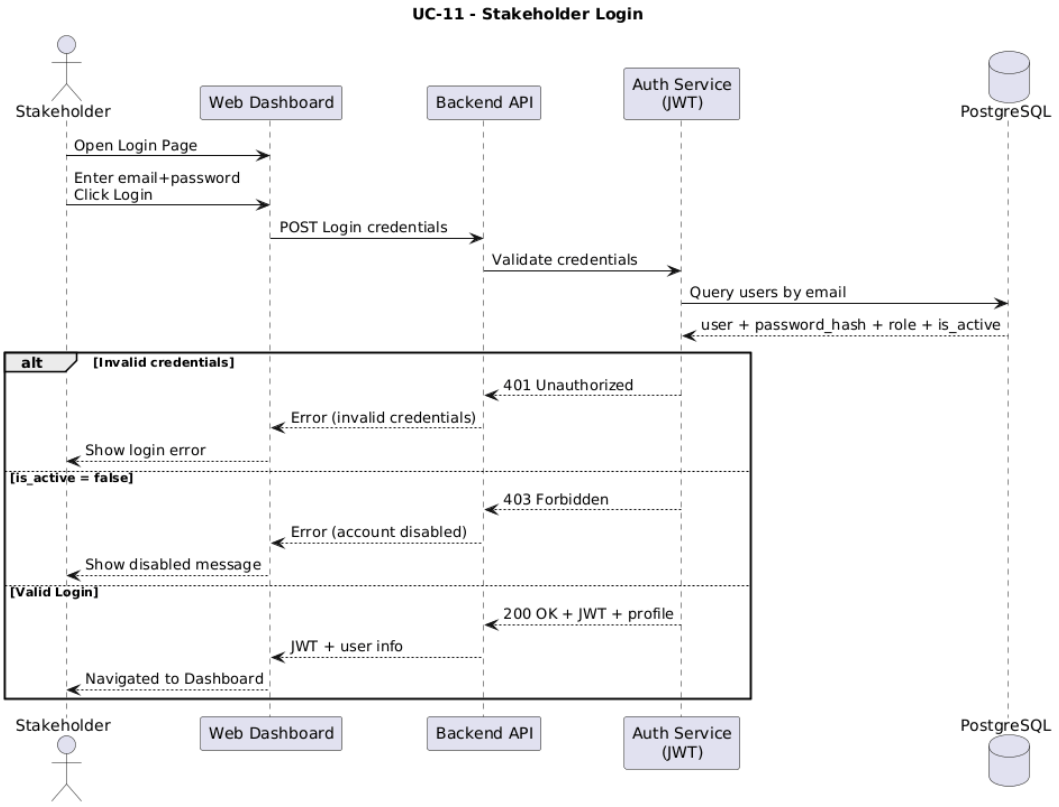
UC-08 - Upload Plant Image



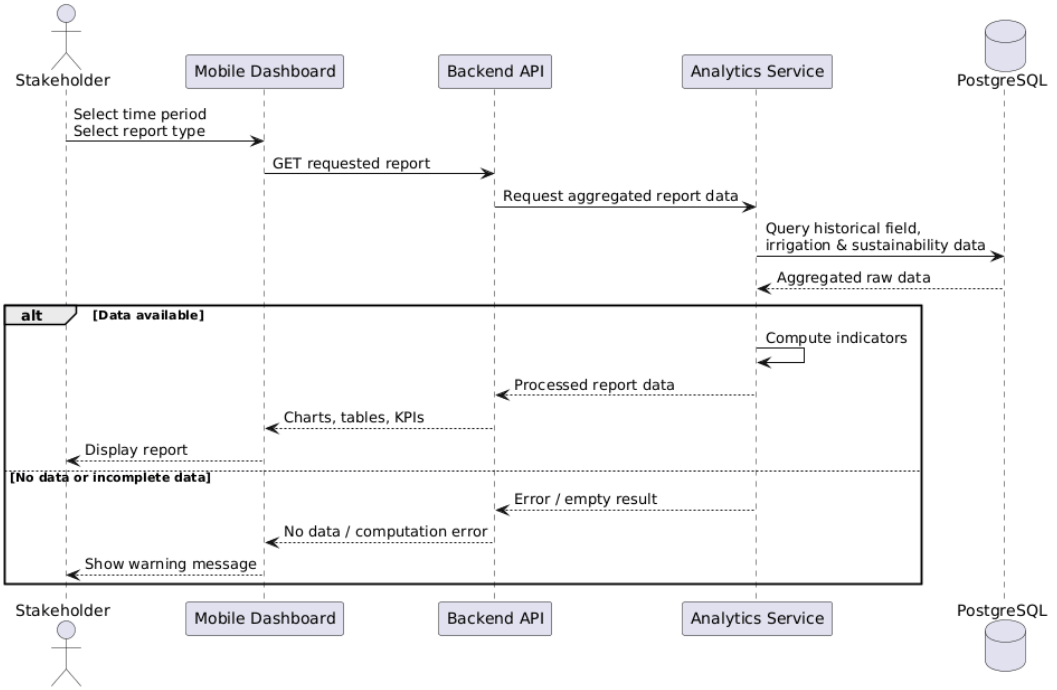
UC-09 - Receive Disease Diagnosis



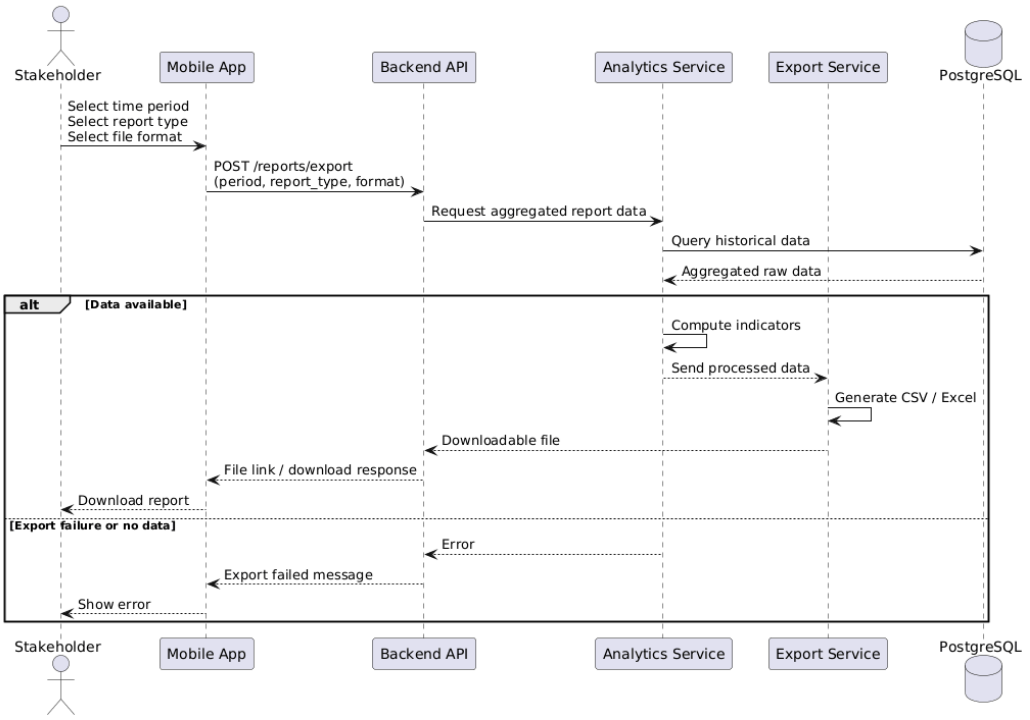

UC-10 - View Recommended Actions
Farmer
Mobile App
Backend API
PostgreSQL
LLM Service
Open "Recommended Actions"
GET plant health result data
Load diagnosis + recommendation rules
diagnosis + mapped actions + warnings
alt
[No recommendation exists]
Empty state
Show "No mapped actions"
[Recommendation exists]
Generate actions summary
(diagnosis + rules + field context)
alt
[LLM is Unavailable]
500: Internal Server Error
Fallback: raw action list + warnings
Show steps
within a basic structure
[Success]
Personalized action plan
(steps, priority, cautions)
return LLM action plan
Display explained recommendations


UC-11 - Stakeholder Login
Stakeholder
Web Dashboard
Backend API
Auth Service
(JWT)
PostgreSQL
Open Login Page
Enter email+password
Click Login
POST Login credentials
Validate credentials
Query users by email
user + password_hash + role + is_active
alt
[Invalid credentials]
401 Unauthorized
Error (invalid credentials)
Show login error
[is_active = false]
403 Forbidden
Error (account disabled)
Show disabled message
[Valid Login]
200 OK + JWT + profile
JWT + user info
Navigated to Dashboard

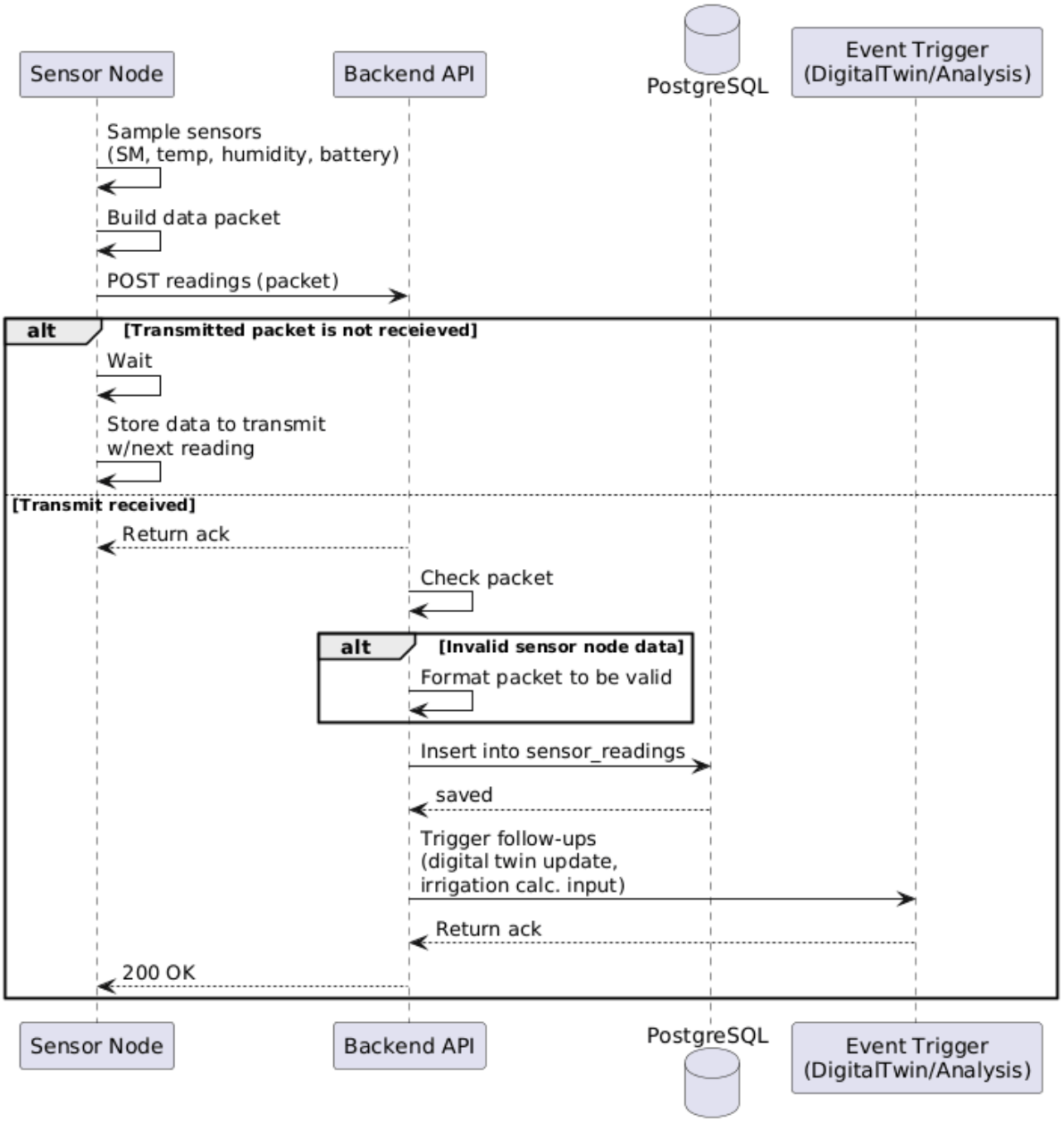
UC-12 - View Sustainability and Carbon Footprint Reports



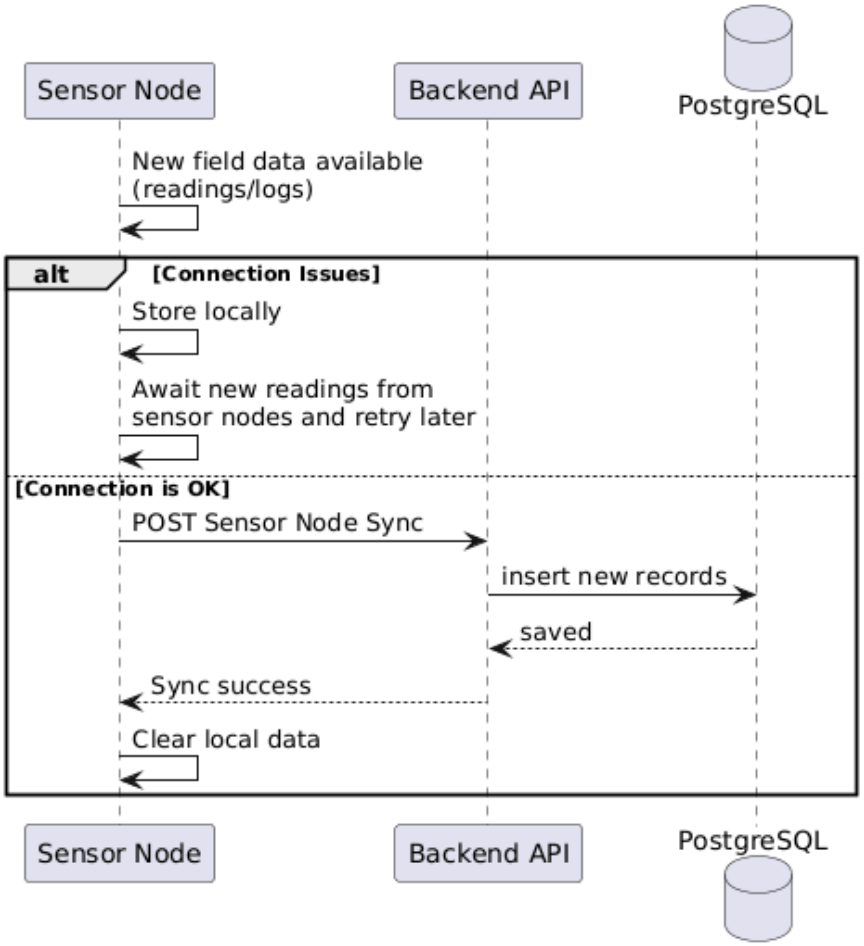
UC-13 - Export Sustainability and Carbon Footprint Reports



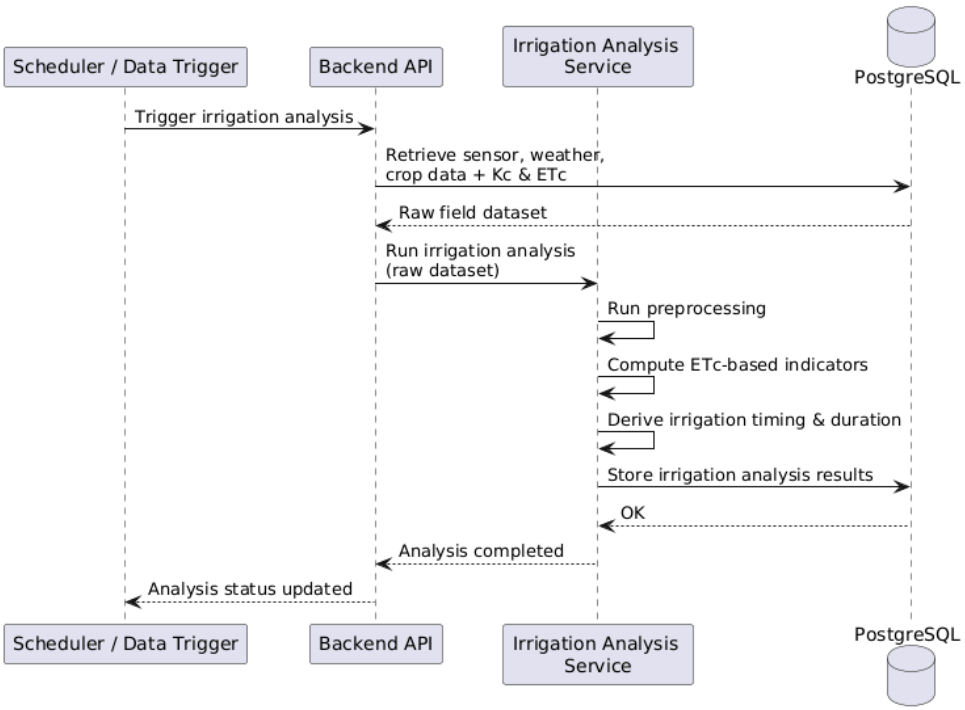
UC-14 - Collect and Transmit Sensor Data



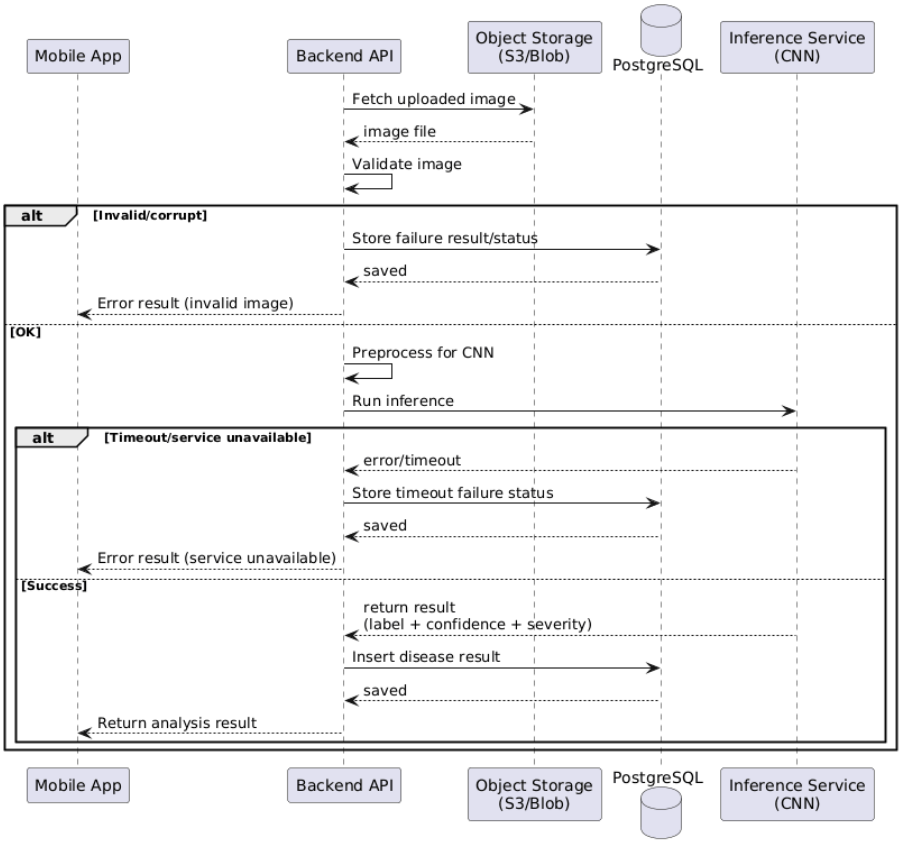
UC-15 - Sync Field Data to Backend



UC-16 - Run Irrigation Analysis



UC-17 - Analyze Plant Image for Diseases



4. Methodology

4.1. Irrigation Prediction

This study does not employ a data-driven ML model for irrigation recommendation. Instead, the methodology is based on a physics- and agronomy-driven approach that combines reference evapotranspiration (ET_0), crop coefficients (Kc), and field sensor observations. Optional data-driven calibration steps (e.g., irrigation gain tuning) are treated as parameter adjustment rather than predictive ML.

4.1.1. Problem Formulation

The irrigation recommendation problem is formulated as a timing and duration estimation task. Given weather conditions, crop characteristics and soil moisture dynamics, the objective is to determine when irrigation should be initiated, based on estimated crop water depletion derived from ETc, and how long irrigation should be applied, by converting the required water amount into pump operation time using irrigation gain parameters.

The core formulation relies on estimating crop evapotranspiration (ETc), defined as the actual water consumption of a crop under current atmospheric conditions and growth stage.

4.1.2. Data Acquisition and Preprocessing

Data required for irrigation estimation are collected from heterogeneous sources:

- Meteorological data (temperature, humidity, wind speed or equivalent variables) used to compute or directly obtain ET_0.
- Field sensor data, primarily soil moisture, supported by air temperature and humidity.
- Crop metadata, including crop type and coefficient (Kc).
- Historical irrigation records, including irrigation events and pump duration or flow-rate information for calibration and validation.

Prior to irrigation estimation, all acquired data undergo a structured preprocessing pipeline:

- Timestamp standardization and alignment across all data sources.
- Unit normalization and physical range validation.
- Sensor quality control, including noise reduction, spike detection and drift monitoring.

- Missing data analysis with context-aware handling based on data criticality.
- Context-aware outlier detection distinguishing valid irrigation-induced changes from sensor anomalies.

4.1.3. Algorithmic Approach

The irrigation recommendation algorithm follows a deterministic and interpretable workflow:

- **ET_0 Estimation:** Reference evapotranspiration is obtained directly or computed from meteorological inputs.
- **Kc Application:** Crop-specific Kc values are generated as a time series based on crop type and growth stage and calibrated through the process.
- **ETc Computation:** Crop evapotranspiration is calculated as **$ETc = ET_0 \times Kc$** [1].
- **Water Depletion Assessment:** Cumulative ETc is evaluated over a rolling time window to estimate soil water depletion.
- **Irrigation Timing Estimation:** Irrigation start time is determined when depletion exceeds predefined or calibrated thresholds.
- **Irrigation Duration Estimation:** Required water depth is converted into irrigation duration using irrigation gain.

4.1.4. System Evaluation Metrics

System performance is evaluated using domain-relevant and operational metrics rather than classification accuracy:

- Consistency between predicted irrigation timing and observed soil moisture recovery
- Deviation between estimated and observed irrigation duration
- Reduction in over-irrigation or under-irrigation events

Where historical irrigation data are available, predicted and actual irrigation durations are compared for validation.

4.1.5. Implementation Details

The irrigation estimation logic is implemented as a modular service within the system architecture. ET_0 computation, Kc profile generation, ETc calculation and irrigation duration estimation are implemented as separate functional modules to ensure maintainability and extensibility.

4.2. Disease Detection

4.2.1. Problem Formulation

This module is designed as an image processing and ML component of the TARAS system, focusing on the analysis of plant leaf images for disease identification. The primary objective is to enable users to capture or upload a leaf image, after which the system automatically detects the presence of plant disease and classifies its type.

Formally, the task is modeled as a multi-class image classification problem. Each input image **X** is mapped to a disease label **Y**, selected from a predefined set of categories representing both healthy and diseased plant conditions. The model aims to achieve high classification accuracy while ensuring robust generalization performance on previously unseen data.

4.2.2. Data Acquisition and Preprocessing

4.2.2.1. Data Acquisition

Plant leaf images are obtained either by direct capture using camera-enabled devices or by user upload through the TARAS system interface. This approach enables the module to operate under real field conditions as well as controlled analysis environments. The acquired images constitute the raw input of the image processing pipeline.

4.2.2.2. Preprocessing

All input images are converted to RGB format to ensure consistency across different devices and imaging conditions. The images are then resized to a fixed resolution of 256 × 256 pixels to enable uniform processing and computational efficiency. Pixel values are normalized using the EfficientNetV2 preprocessing function to align the input distribution with ImageNet-pretrained weights.

4.2.3. Algorithmic Approach

The disease detection module employs a deep learning–based image classification approach to identify plant diseases from leaf images. Convolutional neural networks (CNNs) are used to automatically learn discriminative visual features from preprocessed images, including texture patterns, color distributions, and structural irregularities that are commonly associated with diseased plant leaves.

Input images are processed through a CNN backbone that transforms raw pixel data into high-level feature representations. These features

enable the model to distinguish between healthy and diseased leaf conditions as well as different disease categories. The use of pretrained CNN architectures allows the system to leverage general visual knowledge learned from large-scale datasets, improving robustness under varying imaging conditions such as illumination changes, background noise, and leaf orientation.

The final classification is performed by a task-specific output layer that maps the extracted features to predefined disease classes. The predicted disease label is selected based on the highest class probability, enabling reliable and automated disease identification within the TARAS system. This approach ensures accurate detection while maintaining adaptability to real-world agricultural scenarios.

4.2.4. Model Selection and Comparative Evaluation

To identify the most suitable architecture for the proposed system, multiple CNN models with varying complexity levels were systematically evaluated, including MobileNetV3-Small, EfficientNet-B0, ResNet50, EfficientNet-B7, and EfficientNetV2-S. All models were trained and tested on the same dataset using identical preprocessing pipelines and evaluation protocols to ensure a fair and consistent comparison.

The quantitative performance comparison of the evaluated architectures on the independent test set is summarized in Table 1. As shown in Table 1, EfficientNetV2-S achieved the highest performance across all major evaluation metrics. Specifically, the model reached an accuracy of 96.55%, a macro F1-score of 96.30%, and a weighted F1-score of 96.54%, outperforming all other evaluated architectures.

Although EfficientNet-B7 is a deeper and more parameter-rich architecture compared to EfficientNetV2-S, its performance on the independent test set was inferior, as summarized in Table 1. This outcome suggests that the increased model complexity did not yield improved generalization for the given dataset and may have led to overfitting effects. In contrast, the lightweight MobileNetV3-Small model achieved competitive accuracy with substantially lower computational cost; however, it did not demonstrate the same level of robustness and consistency as EfficientNetV2-S across precision, recall, and F1-score metrics.

Based on these findings, EfficientNetV2-S was selected as the core classification model due to its superior balance between classification accuracy, generalization performance, and computational efficiency, making it well suited for deployment within the TARAS system.

Model	Accuracy	Precision (Macro)	Recall (Macro)	F1 (Macro)	Precision (Weighted)	Recall (Weighted)	F1 (Weighted)
EfficientNetV2-S(Base Model)	**0.9654**	0.9662	0.9610	0.9629	0.9659	0.9654	0.9653
EfficientNetB0	0.8564	0.8473	0.8192	0.8276	0.8574	0.8564	0.8547
MobileNetV3Small	0.9313	0.9286	0.9069	0.9155	0.9311	0.9313	0.9302
ResNet-50	0.7916	0.7738	0.7564	0.7621	0.7927	0.7916	0.7894
EfficientNetB7	0.9055	0.8958	0.8736	0.8818	0.9066	0.9055	0.9046

Table 1. Test-set performance comparison of the evaluated CNN models using accuracy, precision, recall, and F1-score metrics.

4.2.5. System Evaluation Metrics

Model performance is evaluated using multiple complementary metrics to capture different aspects of classification quality:

- **Accuracy**: Overall proportion of correctly classified images
- **Precision**: Ability of the model to avoid false disease predictions
- **Recall**: Ability of the model to correctly identify diseased samples
- **F1-score**: Harmonic mean of precision and recall

4.2.6. Implementation Details

The plant disease detection module is implemented using the TensorFlow and Keras deep learning frameworks. Model training and evaluation are conducted in an offline environment, while the trained model is deployed for inference as part of the TARAS image processing pipeline. This separation ensures efficient development while enabling real-time disease classification during system operation.

Within the TARAS system, the module receives preprocessed leaf images as input and performs inference using the trained EfficientNetV2-S model. The model outputs a probability distribution over predefined disease classes, and the class with the highest probability is selected as the predicted disease label. This output is then forwarded to higher-level TARAS components to support

decision-making processes such as disease monitoring and agricultural analysis.

The implementation supports modular integration, allowing the disease detection component to operate independently from other system modules. This design facilitates future updates, including model retraining, architecture replacement, or deployment on different computational platforms without affecting the overall system structure.

4.3. LLM-Based Advisory & Decision Support

4.3.1. Problem Formulation

The objective of the LLM-based advisory component is to provide context-aware, human-readable decision support by interpreting outputs generated by the irrigation prediction and disease detection modules. While these modules produce numerical predictions or categorical outputs, end users such as farmers may lack the technical expertise to directly interpret these results. Therefore, the problem is formulated as a natural language generation and reasoning task, where structured system outputs and contextual data are transformed into actionable recommendations and explanations.

Formally, given a set of structured inputs C (input string) representing environmental data, model predictions, and user context, the LLM generates a natural language response R (output string) that explains system outputs and provides advisory guidance aligned with agricultural best practices.

4.3.2. Input Representation and Context Construction

The input to the LLM is constructed as a structured context that combines multiple information sources, including irrigation predictions, detected disease classes, crop type, growth stage and recent environmental conditions.

Context construction follows a deterministic process in which numerical outputs are normalized, categorical variables are mapped to domain-specific terminology and temporal information is summarized. This structured context is embedded within a prompt that constrains the LLM to generate responses grounded in system data, thereby minimizing hallucinations and ensuring domain relevance.

4.3.3. Response Generation and Explainability

The LLM generates clear and actionable natural-language advisory responses by explaining underlying system outputs and explicitly referencing key input factors such as soil moisture, growth stage, or

visual symptoms. This approach improves transparency and user trust by clarifying not only what is recommended, but also why the recommendation is made.

4.3.4. System Evaluation and Limitations

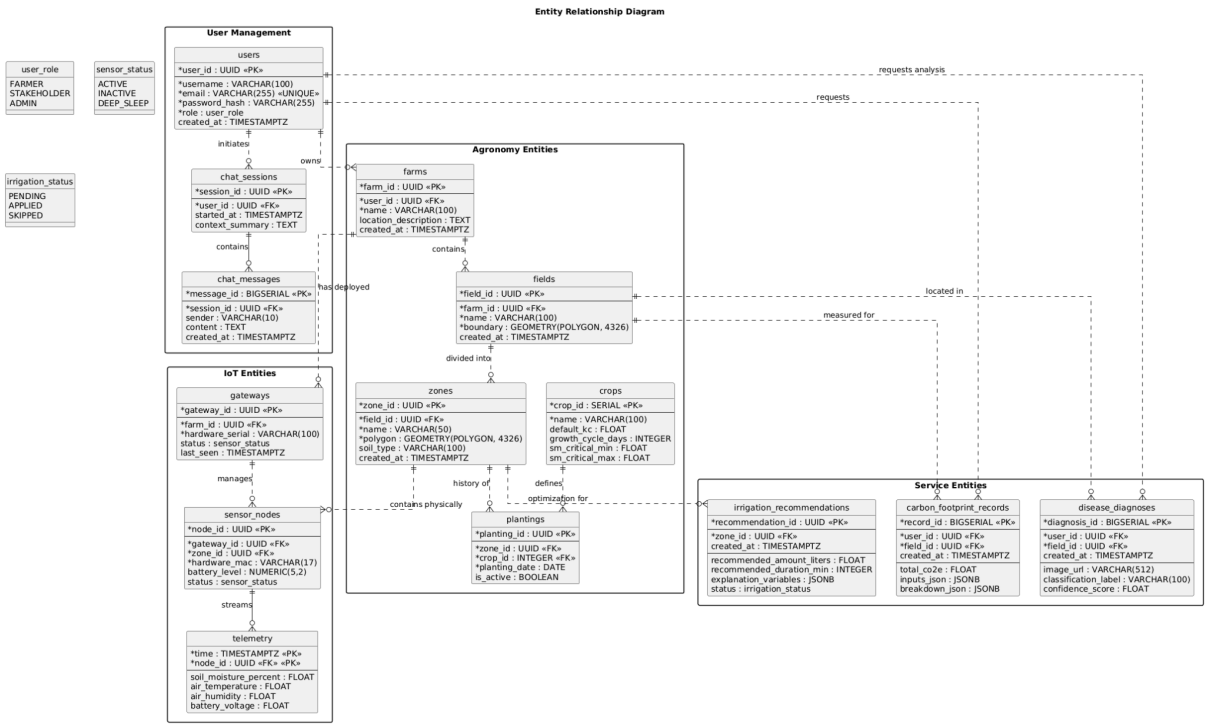
The LLM-based advisory system is evaluated qualitatively based on explanation coherence, consistency with system outputs and usability from an end-user perspective. As it does not perform prediction tasks, the evaluation focuses on the correctness and domain alignment of generated explanations. The system is inherently dependent on the quality of upstream model outputs and prompt design, and its reliability may be reduced for unseen crops or disease types; therefore, the advisory component is intended strictly as a decision-support aid and does not replace expert agronomists.

4.3.5. Implementation Details

The LLM-based advisory module is implemented as a cloud-based service integrated with the TARAS backend. It receives structured inputs via an API interface and returns generated responses to the user interface layer

5. Data Design

5.1. Entity-Relationship Diagram



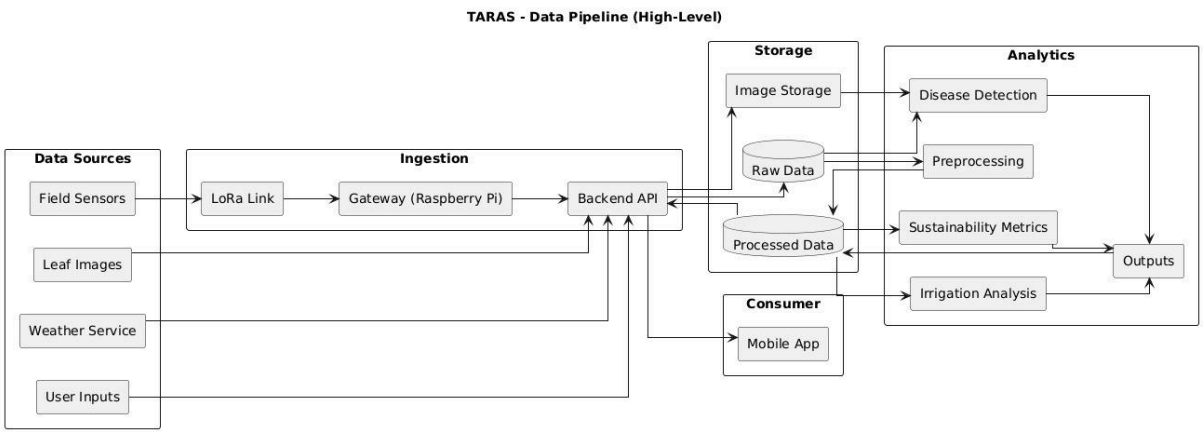
5.2. Data Pipeline

The TARAS data pipeline defines how heterogeneous data collected from field sensors, user inputs, and image uploads are transformed into reliable and analysis-ready information. The pipeline is designed to support real-time monitoring, historical analysis, and decision-support operations while maintaining data consistency and traceability.

Field sensor measurements such as soil moisture and environmental conditions are transmitted to gateway devices and forwarded to cloud services. In parallel, meteorological data, crop metadata, irrigation parameters and plant images are received through backend interfaces. All incoming data are timestamped and associated with field and device identifiers to ensure coherent integration.

Before storage, the data undergo preprocessing steps including timestamp alignment, unit normalization, basic quality checks, and context-aware filtering to handle missing values and abnormal sensor readings. Processed numerical data are stored in a centralized database, while image data are stored in dedicated storage buckets with linked metadata.

The processed data are then consumed by analytical modules responsible for irrigation estimation, disease detection, and sustainability analysis. Derived outputs such as irrigation recommendations, classification results and environmental indicators are stored as processed records and delivered to the mobile application through backend APIs. This structured pipeline enables consistent data flow across sensing, analytics and visualization layers and provides a scalable foundation for the TARAS platform.



6. User Interface Design

6.1. Interface Hierarchy

The interface follows a hierarchical structure that prioritizes critical agricultural information over details. Core metrics such as soil moisture, temperature, and field status are positioned at the highest level of the interface hierarchy and are intended to be presented through large, clearly separated data cards. This structure enables users to access essential information with minimal navigation and reduces cognitive effort during decision-making.

The hierarchy is intentionally simplified to accommodate the diverse demographic profile of farmers, particularly older individuals with limited technical experience. By minimizing nested menus and reducing visual competition between interface elements, the interface supports rapid identification of actionable insights without overwhelming the user.

6.2. Visual Presentation

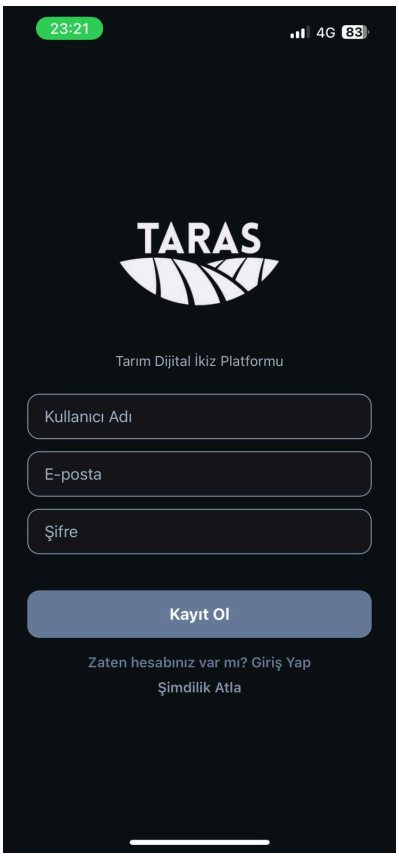
A balanced and minimalistic color palette was adopted as the foundation of the visualization and interaction layer. The palette is deliberately kept simple so that digital twin visualizations remain visually distinct from the interface background across different contexts.

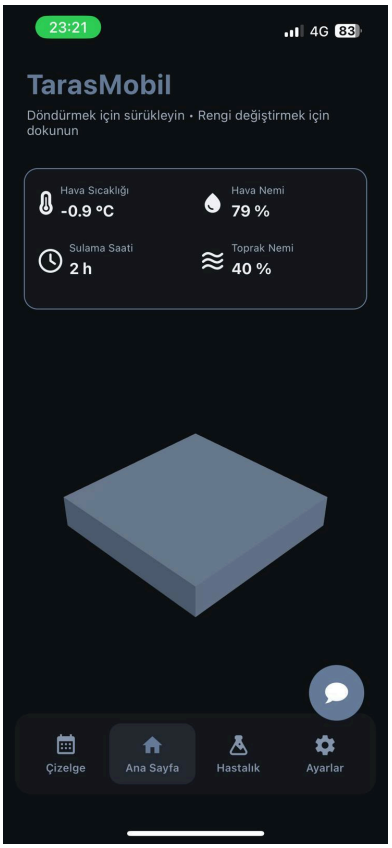
Light and dark modes are defined to assign inverse roles to background and text elements, preserving foreground and background separation, and visual comfort under varying lighting conditions.

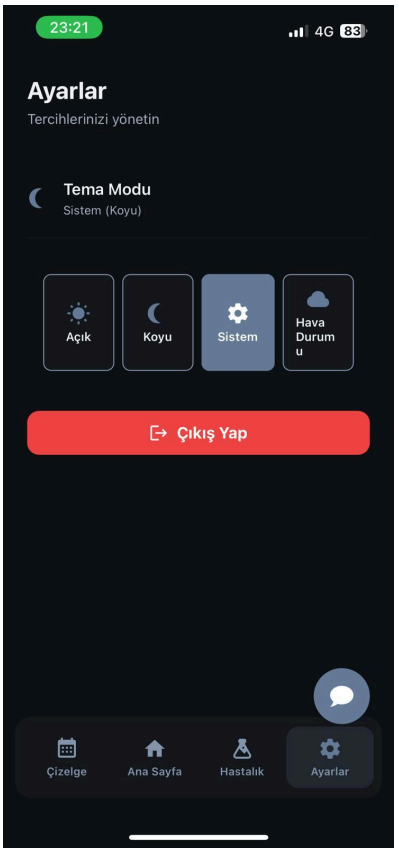
6.3. Accessibility and Responsivity

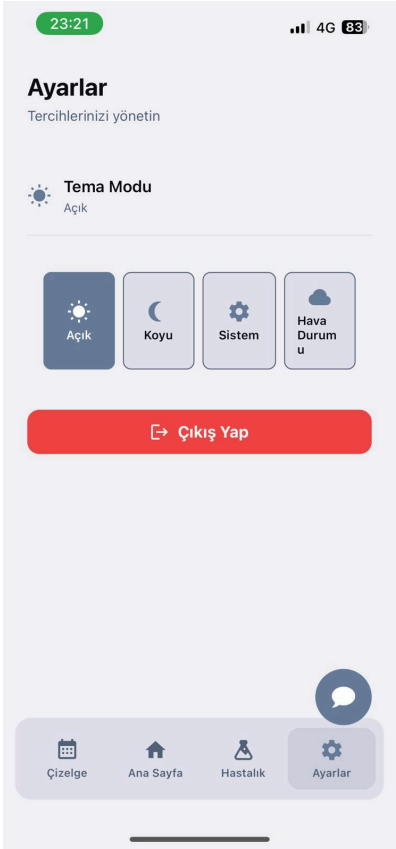
The interface design adheres to web accessibility and readability principles, including WCAG contrast recommendations, to support usability across varying lighting conditions and display settings. Text sizes, spacing, and contrast levels are specified to compensate for age-related declines in visual acuity and cognitive flexibility.

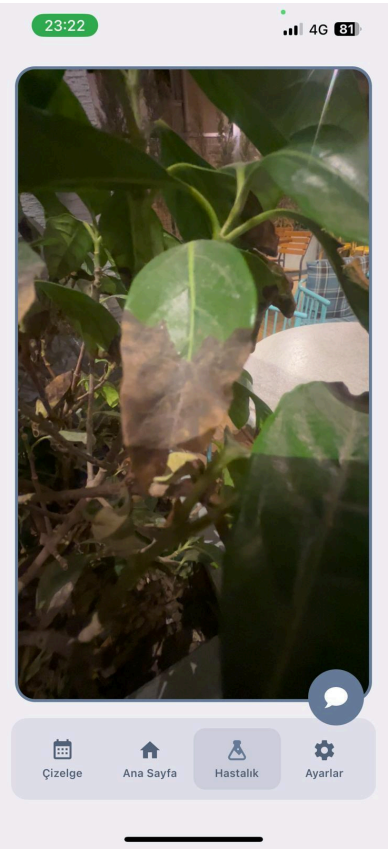
To further reduce cognitive load, essential information is structured to be presented in a concise and visually separated manner. In addition, an AI-powered conversational assistant is planned as a core interaction mechanism to support users with varying levels of technological literacy [5].

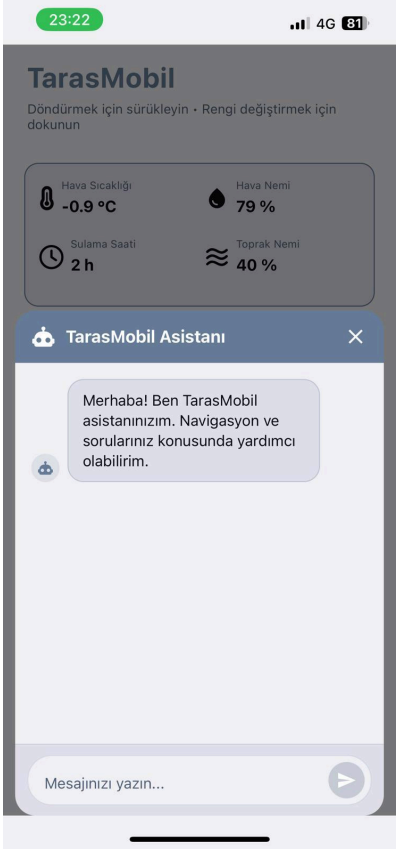

23:21
4G 83
TARAS
Tarım Dijital İkiz Platformu
Kullanıcı Adı
E-posta
Şifre
Kayıt Ol
Zaten hesabınız var mı? Giriş Yap
Şimdilik Atla


23:21
4G 83
TarasMobil
Döndürmek için sürükleyin • Rengi değiştirmek için dokunun
Hava Sıcaklığı
-0.9 °C
Hava Nemi
79 %
Sulama Saati
2 h
Toprak Nemi
40 %
Çizelge
Ana Sayfa
Hastalık
Ayarlar


23:21
4G 83
Ayarlar
Tercihlerinizi yönetin
Tema Modu
Sistem (Koyu)
Açık
Koyu
Sistem
Hava Durum u
Çıkış Yap
Çizelge
Ana Sayfa
Hastalık
Ayarlar


23:21
4G 83
Ayarlar
Tercihlerinizi yönetin
Tema Modu
Açık
Açık
Koyu
Sistem
Hava Durum u
Çıkış Yap
Çizelge
Ana Sayfa
Hastalık
Ayarlar


23:22
4G 81
Çizelge
Ana Sayfa
Hastalık
Ayarlar


23:22
4G 81
TarasMobil
Döndürmek için sürükleyin • Rengi değiştirmek için dokunun
Hava Sıcaklığı
-0.9 °C
Hava Nemi
79 %
Sulama Saati
2 h
Toprak Nemi
40 %
TarasMobil Asistanı
Merhaba! Ben TarasMobil asistanınızım. Navigasyon ve sorularınız konusunda yardımcı olabilirim.
Mesajınızı yazın...

7. Conclusion

This document outlined the system architecture and methodology of the TARAS precision agriculture platform, focusing on its layered structure, modular subsystems and data-driven decision logic. By combining field sensing, agronomy-based irrigation analysis, AI-supported disease detection and user-centered interface design, TARAS provides a scalable and maintainable foundation for intelligent agricultural support. The proposed architecture enables future extensions while ensuring reliable and accessible system operation.

8. References

[1]*Allen, R. G., Pereira, L. S., Raes, D., & Smith, M. (1998).*

Crop evapotranspiration – Guidelines for computing crop water requirements.

FAO Irrigation and Drainage Paper No. 56.

Food and Agriculture Organization of the United Nations.

https://www.fao.org/4/x0490e/x0490e00.htm

[2]Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016).
Using deep learning for image-based plant disease detection.
https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2016.01419/full

[3] Tan, M., & Le, Q. (2019).
EfficientNetV2: Smaller Models and Faster Training
Proceedings of the 36th International Conference on Machine Learning (ICML)
https://proceedings.mlr.press/v139/tan21a/tan21a.pdf

[4]*Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014).*

How transferable are features in deep neural networks?

Advances in Neural Information Processing Systems (NeurIPS).

https://arxiv.org/abs/1411.1792

[5] Czaja, S. J., & Lee, C. C. (2007).

The impact of aging on access to technology.

Universal Access in the Information Society, 5(4), 341–349.

https://www.researchgate.net/publication/220606716_The_impact_of_aging_on_access_to_technologdr